

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

GILMAR GOMES DO NASCIMENTO

O USO DE LOGS NO ENSINO DE PROGRAMAÇÃO

CURITIBA

2026

GILMAR GOMES DO NASCIMENTO

O USO DE LOGS NO ENSINO DE PROGRAMAÇÃO

USING LOGS TO SUPPORT PROGRAMMING EDUCATION

Proposta de Dissertação apresentado como requisito para obtenção do título de Mestre em Ciência da Computação do Mestre em Computação Aplicada da Universidade Tecnológica Federal do Paraná.

Orientador(a): Prof. Dr. Laudelino Cordeiro Bastos

Coorientador(a): Profa. Dra. Maria Claudia Figueireido Pereira Emer

CURITIBA

2026



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

AGRADECIMENTOS

Agradeço imensamente à minha família e amigos pelo apoio e paciência que tiveram comigo durante esse longo período.

Agradeço imensamente ao Instituto Federal de Educação, Ciência e Tecnologia do Amazonas, em especial ao campus avançado Boca do Acre pela oportunidade de realizar essa pesquisa no meu dia a dia de trabalho e fornecer todos os recursos necessários.

Por último, agradeço aos meus orientadores Laudelino Cordeiro Bastos, Maria Claudia Emer e Adolfo Gustavo Serra Seca Neto, pois, sem a orientação e paciência deles, este trabalho não teria sido possível.

Enfim, a todos os que por algum motivo contribuíram para a realização desta pesquisa.

RESUMO

In software development, metrics are commonly used to assess code quality and development time. Professionally, this is often done via pull requests, whereas in academia, assessment relies on occasional submissions through emails or Learning Management Systems (LMS). Although LMSs record activity logs, a significant gap remains in the structured collection of data on the programming learning process, particularly in computer lab environments. This work proposes creating a dataset of code development logs to capture the learning trail in real-time. Unlike conventional approaches that rely on LMSs, data will be collected via a plugin integrated into a widely used code editor. The resulting database is expected to help identify student difficulties, enable personalized teaching, and improve pedagogical practices.

Palavras-chave: ensino de programação; log; telemetria; logging.

ABSTRACT

In software development, metrics are commonly used to assess code quality and development time. Professionally, this is often done via pull requests, whereas in academia, assessment relies on occasional submissions through emails or Learning Management Systems (LMS). Although LMSs record activity logs, a significant gap remains in the structured collection of data on the programming learning process, particularly in computer lab environments. This work proposes creating a dataset of code development logs to capture the learning trail in real-time. Unlike conventional approaches that rely on LMSs, data will be collected via a plugin integrated into a widely used code editor. The resulting database is expected to help identify student difficulties, enable personalized teaching, and improve pedagogical practices.

Keywords: programming education; log; telemetry; logging.

LISTA DE FIGURAS

Figura 1 – A posição dos logs na relação entre ensino (input) e produção discente (output)	9
Figura 2 – Arquitetura de coleta e agregação de logs pelo plugin LOGADO.Fonte: Elaborado pelo autor.	10
Figura 3 – Modelo de coleta de dados educacionais durante a atividade de codificação. O plugin LOGADO monitora o processo, gerando logs que registram a jornada do aprendiz, em contraste com o artefato final (código) que é o foco da avaliação tradicional.	18
Figura 4 – Ícone da extensão LOGADO	19
Figura 5 – Visual Studio Code com a extensão LOGADO instalada.....	19
Figura 6 – Notificação exibida pela extensão LOGADO ao ser ativada	21
Figura 7 – Arquivos de log (. json) gerados para cada arquivo JavaScript	21
Figura 8 – Casos de uso do <i>plugin</i> LOGADO - Visão do Discente	26
Figura 9 – Casos de uso do <i>plugin</i> LOGADO - Visão do Pesquisador.....	26
Figura 10 – Top 20 palavras-chave mais frequentes nos logs coletados	31
Figura 11 – Distribuição dos logs por arquivo	32
Figura 12 – Ocorrência de logs por hora do dia	33
Figura 13 – Rede de co-ocorrência entre palavras reservadas	34
Figura 14 – Palavras com maior centralidade na rede de co-ocorrência	35
Figura 15 – Palavras com maior crescimento ao longo das semanas	36
Figura 16 – Fluxograma PRISMA do processo de seleção de estudos	50
Figura 17 – Distribuição das características principais dos estudos incluídos (n=17)	52
Figura 18 – Instituições que participaram do formulário	58
Figura 19 – Mapa de calor representando a frequência de uso e familiaridade com diferentes ferramentas tecnológicas (Perguntas 2 e 3). As cores mais quentes indicam maior frequência de uso ou familiaridade, permitindo identificar quais ferramentas são mais dominadas pelos participantes da pesquisa.	59
Figura 20 – Gráfico Waffle comparando características ou percepções entre diferentes grupos de participantes (Perguntas 4 e 5). Cada célula representa uma unidade de resposta, permitindo visualizar proporções e distribuições de forma intuitiva.	60
Figura 21 – Gráfico de barras agrupadas com facetas mostrando a distribuição de respostas sobre uso de datasets e sistemas de correção automatizada (Perguntas 6 e 7). As facetas permitem comparar subcategorias de respostas simultaneamente.	60
Figura 22 – Gráfico de pontos representando a relação entre a percepção sobre plugins educacionais e sua frequência de uso (Perguntas 9 e 11). A posição dos pontos indica a relação entre estas duas variáveis, podendo sugerir correlações.....	61
Figura 23 – Distribuição de IDEs utilizadas pelas instituições	62

SUMÁRIO

1	INTRODUÇÃO	8
1.1	Contexto, motivação e justificativa	11
1.2	Questões de pesquisa.....	12
1.3	Objetivos	12
1.3.1	Objetivos Gerais	12
1.3.2	Tarefas Metodológicas	13
1.4	Contribuições esperadas	13
1.5	Estrutura do trabalho	14
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	Desafios no Ensino de Programação e a Lacuna Instrucional sobre Log- ging	16
2.1.1	Pesquisas sobre Logs no Ensino de Programação no Brasil	16
2.2	Logging: Conceitos e Aplicações na Engenharia de Software	16
2.3	A Análise de Logs como Ferramenta Educacional	17
2.4	Design Science Research.....	18
2.5	A Extensão LOGADO: Proposta de Implementação	19
2.5.1	Estado atual da implementação.....	19
2.5.2	Funcionalidades Principais	20
2.5.3	Arquitetura e Armazenamento de Dados	20
2.5.4	Notificação de Ativação	20
2.5.5	Geração de Logs	21
3	METODOLOGIA	23
3.1	Abordagem Metodológica	23
3.1.1	Revisão Sistemática da Literatura	23
3.1.2	Survey Exploratório.....	23
3.1.3	Design Science Research	24
3.1.3.1	Atividade 1: Identificação do Problema e Motivação.....	24
3.1.3.2	Atividade 2: Definição dos Objetivos da Solução	24
3.1.3.3	Atividade 3: Design e Desenvolvimento	25
3.1.3.4	Especificação de Requisitos do Artefato	25
3.1.3.4.1	<i>Requisitos Funcionais.....</i>	<i>27</i>
3.1.3.4.2	<i>Requisitos Não-Funcionais</i>	<i>27</i>
3.1.3.4.3	<i>Desenvolvimento do Artefato.....</i>	<i>27</i>
3.1.3.5	Atividade 4: Demonstração e Avaliação	28
3.1.3.5.1	<i>Contexto e Participantes</i>	<i>28</i>
3.1.3.5.2	<i>Procedimentos de Coleta</i>	<i>28</i>
3.1.3.5.3	<i>Avaliação do Artefato.....</i>	<i>28</i>
3.2	Plano de Análise de Dados	28
3.2.1	Pré-processamento e Limpeza.....	28
3.2.2	Análise Exploratória e Identificação de Padrões	29
3.2.3	Potencial do Dataset e Possibilidades de Análise.....	29
3.3	Percurso de Execução da Pesquisa	29
4	RESULTADOS E DISCUSSÕES	31
4.1	Caracterização da amostra	31
4.1.1	Estrutura do dataset gerado	37
4.1.2	Contribuições para a telemetria educacional	38

4.1.3	Limitações contextuais: a contenção de estudantes.....	39
4.1.4	Comportamento observado: interação dos estudantes com os logs	39
5	CONSIDERAÇÕES FINAIS	40
5.0.1	Limitações do estudo.....	40
5.0.2	Trabalhos Futuros.....	41
6	CONCLUSÃO	43
	REFERÊNCIAS	44
	APÊNDICE A MAPEAMENTO SISTEMÁTICO DA LITERATURA.....	48
	A.1 Objetivo e questões de pesquisa	48
	A.2 Estratégia de busca e repositórios	48
A.2.1	Repositórios consultados	48
A.2.2	Strings de busca	48
A.2.3	Idioma dos trabalhos	49
	A.3 Critérios de inclusão e exclusão	49
A.3.1	Critérios de inclusão	49
A.3.2	Critérios de exclusão	49
	A.4 Resultados da busca e seleção de estudos.....	50
	A.5 Caracterização dos estudos incluídos	50
	A.6 Respostas às questões de pesquisa	52
A.6.1	QP1: Quais são as principais aplicações do uso de logs no ensino de programação?	52
A.6.2	QP2: Quais ferramentas, métodos e métricas são mais utilizados para capturar, armazenar e analisar logs em ambientes de ensino de programação?.....	54
	A.7 Lacunas identificadas e trabalhos futuros	55
	APÊNDICE B SURVEY	57
	B.1 Método	57
	B.2 Instrumento de pesquisa	57
	B.3 Respostas	58
B.3.1	Análise de Ferramentas Utilizadas	59
B.3.2	Comparações entre Grupos	60
B.3.3	Análise de Dataset e Correções	60
B.3.4	Opinião sobre Plugins no Ensino	61
B.3.5	IDEs utilizados por Instituições.....	61
	APÊNDICE C ESTRUTURA DO DATASET GERADO	64

1 INTRODUÇÃO

Enquanto o ensino tradicional foca na leitura e escrita de textos, os cursos de programação introduzem uma nova forma de pensamento: os algoritmos. Estes são sequências lógicas interpretadas por humanos e máquinas. Essa disciplina, presente em cursos de áreas tecnológicas, é “considerada desafiadora para alunos e professores” (Raabe; Silva, 2005), pois exige o desenvolvimento de uma lógica formal.

Ainda segundo (Raabe; Silva, 2005), as dificuldades no aprendizado de programação podem ser categorizadas em três grupos distintos, que vão além da simples aptidão individual: Problemas de natureza didática (relacionados aos métodos de ensino), cognitiva (relacionados aos desafios de raciocínio lógico e abstração) e afetiva (relacionados à motivação, ansiedade e confiança do aluno). Essa categorização sistêmica ajuda a entender que o desafio requer iniciativas diferentes.

No contexto educacional, a avaliação é entendida como uma etapa do processo de ensino e aprendizagem, o que a torna um tema permanente de reflexão e estudo entre especialistas da área, que buscam constantemente refinar suas práticas para alcançar melhores resultados (Lima *et al.*, 2022).

A avaliação no curso técnico configura-se como um processo bifronte, pois contempla tanto a avaliação da aprendizagem do estudante quanto a avaliação do sistema educacional como um todo. No que se refere à aprendizagem, o projeto pedagógico estabelece como objetivo central a progressão do discente para o alcance do perfil profissional almejado. Nesse sentido, o Art. 34º da Resolução CNE/CEB nº 6, de 20 de setembro de 2012, que rege o curso, determina que esse processo “é contínuo e cumulativo, com prevalência dos aspectos qualitativos sobre os quantitativos, bem como dos resultados ao longo do processo sobre os de eventuais provas finais” (IFAM, 2020).

Paralelamente, a avaliação do sistema, referente ao rendimento acadêmico, deve ser realizada para cada disciplina individualmente, considerando de forma simultânea e obrigatória a frequência do aluno e seu aproveitamento na aquisição de competências e conhecimentos (IFAM, 2020).

O ensino de programação é uma atividade multidisciplinar que demanda uma práxis pedagógica apurada — ou seja, uma constante reflexão que integre teoria e prática em sala de aula. Uma estratégia comum é utilizar exemplos do cotidiano para estimular a resolução de problemas por meio de algoritmos. Contudo, avaliar a qualidade desses algoritmos exige ir além da verificação funcional.

Esses algoritmos, definidos como procedimentos computacionais que transformam entradas em saídas, geram registros conhecidos como logs. Mas como esses logs podem ser utilizados para transformar o ensino e a aprendizagem? Este texto explora a interseção entre tecnologia e educação, mostrando como a análise de logs pode oferecer perspectivas valiosas para personalizar o ensino e identificar dificuldades dos alunos.

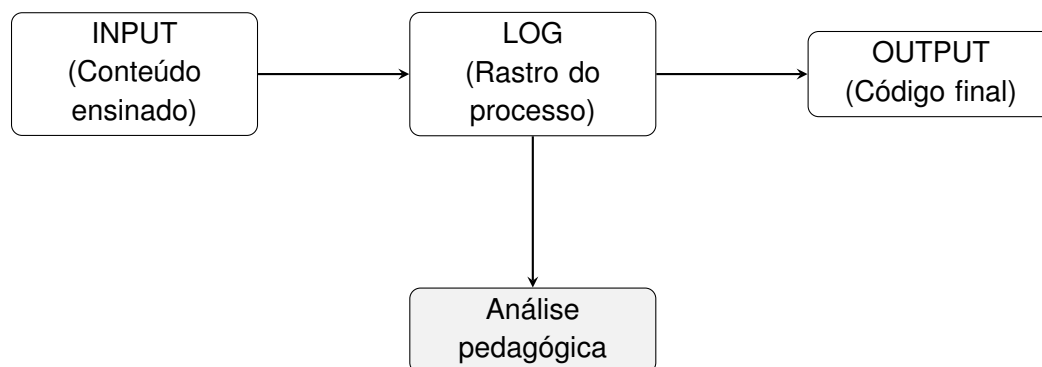


Figura 1 – A posição dos logs na relação entre ensino (input) e produção discente (output)

Esses registros, ou logs, são gerados continuamente durante as interações entre humanos e máquinas. Conforme trazido por (Clemente, 2008), “log é definido como um conjunto de registros com marcação temporal, que suporta apenas inserção, e que representa eventos que aconteceram em um computador ou equipamento de rede”. Além disso, segundo (Gu *et al.*, 2022), “um log geralmente contém um grande número de entradas, também conhecidas como mensagens de registro”. Em geral, o nível do log, seu conteúdo e o local adequado para declará-lo são decisões tomadas pelos desenvolvedores durante a criação do software.

O processo de ensino pode ser compreendido analogamente a uma tecnologia que pode ser atualizada, adaptada ou aplicada em diferentes contextos. No entanto, para estimar o tempo de produção de um código, métodos tradicionais de marcação de tempo são insuficientes, sendo necessárias métricas adicionais. Nesse sentido, o armazenamento de logs permite coletar informações detalhadas sobre eventos em dispositivos. Como destacado por (Nguyen; Ha; Chen, 2023), “esse recurso de previsão possibilita que instrutores identifiquem alunos com dificuldades desde o início e ofereçam intervenções direcionadas para apoiar sua jornada de aprendizado”. Ademais, a análise de logs pode revelar padrões de comportamento e envio, fornecendo, assim, dados valiosos não apenas para a intervenção individual, mas para a melhoria contínua e baseada em evidências do processo educacional como um todo.

A coleta automatizada de logs de programação para análise empírica é uma metodologia consolidada em estudos de Engenharia de Software. O projeto Besouro, por exemplo, foi concebido como um instrumento para capturar dados brutos do processo de TDD, com o objetivo de ‘comparar definições existentes, avaliá-las com relação à percepção dos usuários e explorar informações do código para análise posterior’ (Becker *et al.*, 2015). Inspirado por essa abordagem, o presente trabalho propõe a extensão LOGADO, que adapta esse princípio de instrumentação para o contexto educacional. Nosso plugin coleta logs de eventos de digitação (palavras reservadas, timestamps, localização no código). O objetivo principal é registrar os rastros da construção do código para, a partir da análise desses registros, encontrar formas de auxiliar o ensino de programação, oferecendo uma base de dados rica para pesquisas em ensino de computação.

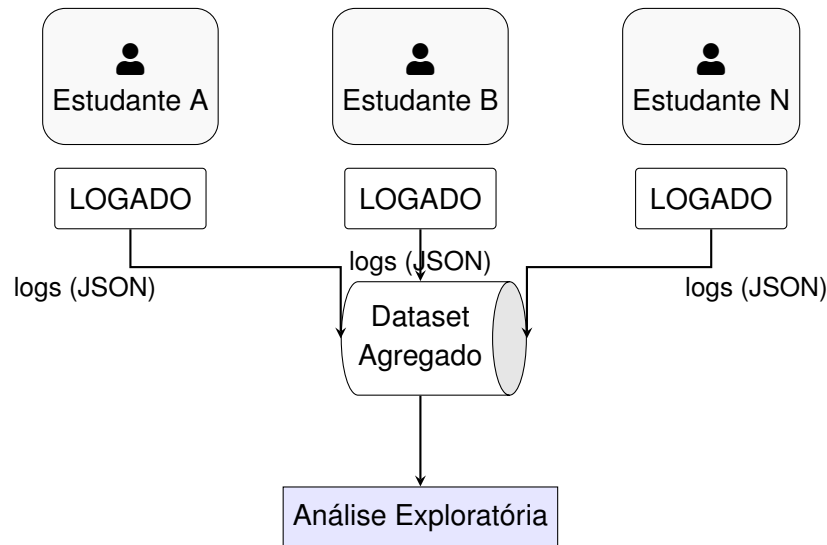


Figura 2 – Arquitetura de coleta e agregação de logs pelo plugin LOGADO.Fonte: Elaborado pelo autor.

A necessidade de analisar o processo de aprendizagem, e não apenas seu produto final, tem sido reconhecida em diferentes domínios da computação. No ensino do paradigma orientado a objetos, por exemplo, (Felisbino; Neto; Bastos, 2018) desenvolveram uma ferramenta com um gerador de logs como plugin para o Microsoft Visio, justamente para capturar as ações dos alunos durante a construção de diagramas de classes e identificar suas dificuldades conceituais. Este trabalho se insere nessa mesma linha de investigação, transladando o princípio da coleta de logs processuais para o domínio do ensino de programação introdutória e de algoritmos.

Diferentemente da análise estática tradicional, que examina o código já pronto, a abordagem proposta registra o processo de construção do código em tempo real. É nesse contexto que métricas baseadas em rastros de construção, aplicadas com um viés pedagógico, se tornam relevantes.

Para além das métricas tradicionais de complexidade algorítmica (como tempo de execução e uso de memória), o docente pode apoiar-se em métricas que avaliam o processo de escrita do código, tais como a sequência de declaração de variáveis, o uso temporal de palavras reservadas e a alternância entre estruturas de controle. Com o suporte do plugin LOGADO, o docente passa a dispor de rastros detalhados do processo de escrita dos estudantes, algo que não seria viável registrar manualmente em um laboratório de informática com a mesma precisão. Esses rastros permitem que o docente identifique padrões de dificuldade e ofereça intervenções mais direcionadas, sem depender exclusivamente da observação presencial imediata.

O potencial inovador dessa abordagem reside na possibilidade de sistematizar a análise: os dados gerados por essas métricas, coletados em larga escala dos códigos dos estudantes, podem ser agregados em um dataset. Este, por sua vez, permite comparações objetivas não apenas entre indivíduos, mas entre turmas, períodos letivos e diferentes metodologias de ensino para avaliação em larga escala.

1.1 Contexto, motivação e justificativa

Diferentemente da avaliação quantitativa, que prioriza resultados finais, a avaliação qualitativa é fundamental para aferir o processo de aprendizado. Em cursos de algoritmos, esse acompanhamento processual pode ser significativamente ampliado com o uso da telemetria. Conforme proposto por (Kennedy, 2015), essa abordagem é complementar aos mecanismos tradicionais de devolutiva, como notas em laboratórios, tarefas e exames. A telemetria não os substitui, mas oferece uma camada adicional de dados contínuos e objetivos sobre o comportamento do estudante, enriquecendo a análise do docente.

O valor desses dados é claro: “saber o que acontece dentro de um sistema enquanto ele está em execução é benéfico para fins de depuração, manutenção e análise”(Gatev, 2021). Um log é um registro com marcação temporal que captura eventos de um sistema, formando um rico histórico de execução (Gu *et al.*, 2022). Nosso trabalho armazena logs dos discentes enquanto o código é produzido, salvando esses registros em arquivos externos ao código-fonte. Dessa forma, podemos observar suas estratégias sem comprometer o ambiente de desenvolvimento.

Embora a Mineração de Dados Educacionais tradicional pressuponha bases de dados estruturadas, a coleta por meio de logs nativos de programação, e não através de acessos em LMS, permanece pouco explorada na literatura. Uma das poucas exceções é o trabalho de (Kennedy, 2015), que aborda a captura de telemetria em tutoriais de programação.

A telemetria é amplamente utilizada em ferramentas de desenvolvimento de software, mas sua aplicação no ensino de programação ainda é um campo em aberto. Ferramentas como o LOGADO contribuem para identificar dificuldades de estudantes a partir de dados quantitativos (timestamps) e qualitativos (palavras reservadas, linha e coluna do código). O estudante, como autor do texto performativo e instrucional, produz o código; o professor, como curador, analisa os logs para compreender o processo do aluno. O código-fonte segue como protagonista, mas os logs ganham destaque como deuteragonistas: pouco explorados até agora, esses registros revelam aspectos da codificação que métricas tradicionais deixam passar, possibilitando intervenções mais direcionadas e eficazes.

Conforme destacado por (Qian; Lehman, 2017), “esforços para aprimorar o ensino em ciência da computação estão em andamento, e os docentes enfrentam desafios significativos em disciplinas introdutórias de programação, visando facilitar a aprendizagem dos estudantes”. Esse diagnóstico, embora formulado a partir da realidade universitária, encontra eco nos cursos técnicos em informática ofertados pela EPCT. É diante desse cenário que a coleta e análise de logs de programação se apresenta como uma via promissora: ao registrar timestamps, palavras reservadas e posições no código, é possível acompanhar o processo do estudante e não apenas o resultado final. Pesquisas como essa mostram que universidades e institutos federais, apesar de contextos distintos, compartilham o mesmo desafio e podem atuar juntos na busca por soluções, reforçando que a pesquisa produzida na EPCT dialoga com a produção acadêmica das universidades. Apesar dos esforços institucionais para mitigar esses problemas, disciplinas como Introdução a Algoritmos, Programação Básica, Estrutura de Dados e Programação Orien-

tada a Objetos são frequentemente associadas a altas taxas de dificuldade e evasão. Além dos obstáculos cognitivos, os estudantes frequentemente lidam com questões emocionais, sentindo-se incapazes de dominar os conceitos e desenvolvendo, conseqüentemente, a crença de que são inaptos para a área de informática como um todo.

A coleta e análise de métricas, especificamente logs de atividade como timestamps, palavras reservadas e posições no código, em ambientes de desenvolvimento integrado (IDE) possibilitam avaliações mais robustas do processo de aprendizagem. A opção por um editor amplamente adotado no setor profissional, o Visual Studio Code (VS Code), que lidera o uso com 75,9% dos desenvolvedores segundo o levantamento mais recente da indústria (Overflow, 2025), busca uniformizar as ferramentas utilizadas no contexto acadêmico, reduzindo a distância entre o que se ensina em disciplinas distintas e as exigências do mercado. A solução proposta consiste, portanto, em um plugin de operação transparente para essa plataforma, executando-se em segundo plano sem comprometer a experiência dos usuários finais (discentes e docentes) durante suas atividades de desenvolvimento.

1.2 Questões de pesquisa

1. QP1: Quais são as principais aplicações do uso de logs no ensino de programação?
2. QP2: Quais ferramentas, métodos e métricas são mais utilizados para capturar, armazenar e analisar logs em ambientes de ensino de programação?

Para responder a essas questões, foi conduzido um Mapeamento Sistemático da Literatura (MSL), detalhado no Apêndice A.

1.3 Objetivos

Este trabalho apresenta um plugin para coleta e análise de logs durante o processo de ensino-aprendizagem de programação. A ferramenta, já implementada e testada, fornece dados detalhados sobre os rastros de construção dos estudantes. Os logs coletados permitiram identificar dificuldades, gerar gráficos e observar características do processo de codificação, oferecendo subsídios para a personalização do ensino e para o aprimoramento das práticas pedagógicas. Os dados e a metodologia aqui apresentados também podem servir de referência para novos projetos na área de telemetria educacional.

1.3.1 Objetivos Gerais

Esta pesquisa tem como objetivos gerais:

1. Compreender, por meio da análise de logs acadêmicos, como estudantes de programação constroem suas soluções ao longo do tempo, identificando não apenas o que

produzem, mas como produzem e quais estratégias, hesitações, correções e escolhas estilísticas emergem durante o processo de codificação.

2. Avaliar a viabilidade e os limites do uso de telemetria no contexto de cursos técnicos de programação, investigando que tipo de inferências pedagógicas podem (e não podem) ser extraídas a partir dos rastros digitais coletados pelo plugin LOGADO.
3. Oferecer à comunidade acadêmica um dataset de logs educacionais, coletado em condições reais de ensino, que possa servir de base para novas investigações sobre dificuldades de aprendizagem, padrões de erro e estratégias de resolução de problemas.

1.3.2 Tarefas Metodológicas

Para atingir os objetivos propostos, o trabalho se organiza nas seguintes tarefas metodológicas:

1. Realizar um Mapeamento Sistemático da Literatura para identificar o estado da arte sobre o uso de logs no ensino de programação, mapeando ferramentas, métricas, aplicações e lacunas existentes.
2. Projetar e implementar o plugin LOGADO para o Visual Studio Code, especificando quais eventos serão capturados (teclas, compilações, salvamentos, execuções) e justificando cada escolha com base nos objetivos de análise.
3. Coletar logs em uma disciplina de programação do Instituto Federal do Amazonas, garantindo os aspectos éticos e de privacidade exigidos pela legislação (LGPD) e pelos comitês de ética.
4. Processar e estruturar os dados brutos em um dataset organizado, documentando as decisões de limpeza, anonimização e agregação adotadas.
5. Analisar os logs coletados sob duas perspectivas complementares: (a) uma análise descritiva, identificando padrões gerais de comportamento; (b) uma análise correlacional, explorando relações entre padrões de codificação e medidas de desempenho.
6. Discutir as implicações pedagógicas dos achados, apontando tanto o potencial quanto as limitações da abordagem proposta para a prática docente em cursos de programação.

1.4 Contribuições esperadas

Este trabalho espera contribuir com a área de ensino de programação sob quatro perspectivas complementares: teórica, técnica, empírica e pedagógica.

Contribuição teórica

Um Mapeamento Sistemático da Literatura sobre o uso de logs no ensino de programação, com ênfase na produção nacional. Diferentemente de revisões anteriores, este mapeamento explicita não apenas as aplicações e ferramentas existentes, mas também as lacunas metodológicas e éticas ainda não endereçadas, oferecendo um panorama crítico que pode orientar futuras pesquisas na área.

Contribuição técnica

O desenvolvimento e a disponibilização pública do plugin LOGADO para o Visual Studio Code, sob licença aberta. A contribuição não se limita ao código, mas inclui a documentação das decisões de projeto, dos eventos capturados e das justificativas pedagógicas para cada escolha, permitindo que outros pesquisadores adaptem e repliquem a solução em seus próprios contextos.

Contribuição empírica

A disponibilização de um dataset com logs reais de programação, coletados em condições autênticas de ensino no Instituto Federal do Amazonas. O dataset é anonimizado e estruturado de forma a permitir análises replicáveis, respeitando a LGPD e os princípios da ciência aberta. Acredita-se que esse conjunto de dados possa subsidiar novos estudos sobre padrões de erro, estratégias de resolução e correlações entre comportamento de codificação e desempenho acadêmico.

Contribuição pedagógica

Como contribuição adicional, este trabalho oferece uma reflexão sobre os limites da análise de logs no contexto educacional, discutindo o que a telemetria pode (e não pode) revelar sobre o processo de aprendizagem. Essa discussão pode ser útil para educadores que consideram adotar abordagens semelhantes, ajudando-os a calibrar expectativas e evitar interpretações equivocadas dos dados.

1.5 Estrutura do trabalho

Este capítulo apresentou o contexto, a motivação e a justificativa do trabalho, suas questões de pesquisa, objetivos, contribuições esperadas e a estrutura da dissertação.

O Capítulo 2 expõe a fundamentação teórica, na qual são definidos os termos e conceitos fundamentais para o entendimento da proposta e discutidos os trabalhos correlatos.

O Capítulo 3 descreve a metodologia de pesquisa adotada, incluindo a abordagem metodológica (Mapeamento Sistemático, Survey Exploratório e Design Science Research), o plano de análise de dados e o percurso de execução da pesquisa.

O Capítulo 4 apresenta os resultados obtidos a partir da coleta e análise dos logs, com a caracterização da amostra, a estrutura do dataset gerado e as discussões sobre os padrões identificados.

O Capítulo 5 traz as considerações finais, retomando as contribuições do trabalho, suas limitações e apontando direções para pesquisas futuras.

Ao final, são apresentados os apêndices, que contêm:

- o protocolo completo do Mapeamento Sistemático da Literatura (Apêndice A);
- o instrumento de coleta e os resultados do survey exploratório (Apêndice B);
- a estrutura e uma amostra do dataset gerado a partir dos logs coletados (Apêndice C).

O código-fonte do plugin LOGADO está disponível publicamente no repositório GitHub do autor, em <https://github.com/gilmargn/logado-utfpr-ifam>, e será oportunamente registrado e incluído em versões futuras deste documento, em consonância com os princípios da ciência aberta.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo contextualiza os desafios do ensino de programação e introduz o conceito de *logging* como uma prática crucial da Engenharia de Software. Em seguida, explora o uso de *logs* no contexto educacional, detalha a taxonomia de conceitos relacionados e descreve a extensão LOGADO, contribuição prática deste trabalho, bem como a Design Science Research como abordagem metodológica adotada.

2.1 Desafios no Ensino de Programação e a Lacuna Instrucional sobre Logging

O ensino de programação vai além da transmissão de conhecimentos técnicos, abrangendo boas práticas de desenvolvimento, integridade acadêmica e a compreensão do ciclo de vida do software. No entanto, a literatura pedagógica apresenta uma lacuna significativa nesse aspecto.

O mapeamento do estado da arte, realizado por meio de uma revisão sistemática de livros-texto fundamentais, evidencia essa lacuna. Obras amplamente adotadas internacionalmente, como as de Pressman (2020) e Sommerville (2016), e, no contexto nacional, *Engenharia de Software Moderna* Valente (2020), dedicam capítulos substanciais a temas como testes e qualidade de software. Nenhuma delas, porém, aborda o *logging* de forma pedagógica e estruturada, como uma prática de engenharia com estratégias e boas práticas específicas, o que está alinhado com observações da literatura recente (Gu *et al.*, 2022).

2.1.1 Pesquisas sobre Logs no Ensino de Programação no Brasil

Diante da lacuna identificada, este trabalho busca reforçar o potencial pedagógico dos rastros digitais no ensino de programação. Para fundamentar essa proposta, foi conduzida uma busca sistemática no Portal da Sociedade Brasileira de Computação (SBC) com a string "log" OR "logging" AND "ensino de programação", que retornou um número insignificante de trabalhos, conforme detalhado na Tabela 5 (Apêndice A).

2.2 Logging: Conceitos e Aplicações na Engenharia de Software

As práticas de registro de rastros em sistemas computacionais abrangem diversos conceitos. Como exemplo, a taxonomia de (Gu *et al.*, 2022) define os seguintes elementos fundamentais¹:

¹ Os termos originais em inglês foram mantidos por serem amplamente utilizados na literatura da área de Engenharia de Software e não terem tradução consagrada no português técnico.

Log statement	Log message	Log
Log level	Log content	Log location
Log placement	Logging	Logging practice

Para os fins desta pesquisa, não é necessário aprofundar cada um desses termos; interessa prioritariamente a distinção entre *log* (registro completo) e *log message* (entrada individual), conforme adotado adiante.

Neste trabalho, *log* refere-se ao registro completo das atividades do sistema, enquanto *log message* compreende entradas individuais com marcação temporal que representam eventos específicos durante o desenvolvimento de código.

Segundo (Rice; Borgman, 1983), o monitoramento de sistemas pode ser entendido como um registro automático de transações. A importância dos *logs* é vasta, sendo utilizados em tarefas como detecção de anomalias, depuração, diagnóstico de desempenho e modelagem de comportamento de sistemas (Fu *et al.*, 2014). Estudos como o de (Yang *et al.*, 2021) confirmam que os desenvolvedores os utilizam primariamente para análise de problemas, buscando informações como a propagação de erros, *timestamps* e o *dataflow* entre componentes.

2.3 A Análise de Logs como Ferramenta Educacional

A aplicação de análise de *logs* no ensino de programação representa uma transposição dessa prática da Engenharia de Software para a avaliação do processo de aprendizagem. Para que essa análise seja viável, é necessário um mecanismo de captura dos rastros gerados durante a atividade. A Figura 3 ilustra esse modelo conceitual de coleta.

A Tabela 1 contrasta a abordagem tradicional com a proposta deste trabalho:

Tabela 1 – Comparação entre abordagens de avaliação em programação

Avaliação Tradicional	Avaliação com Análise de Logs
Foca no produto final (código)	Foca no processo de desenvolvimento
Analisa o que foi produzido	Analisa como foi produzido
Avaliação predominantemente quantitativa	Inclui dimensões qualitativas do aprendizado
Visão estática do conhecimento	Visão dinâmica da curva de aprendizagem
Foco no indivíduo	Permite análise individual e em grupo

Enquanto a avaliação tradicional concentra-se na lógica, sintaxe e funcionalidade do código entregue, a análise de *logs* captura os rastros digitais do desenvolvimento, revelando padrões de raciocínio, dificuldades específicas e estratégias de resolução de problemas.

Como sugerido por (Silva *et al.*, 2022), variáveis preditoras podem ser extraídas diretamente do código-fonte, tais como: número de linhas lógicas, uso de estruturas de repetição, quantidade de identificadores e funções utilizadas. Esta abordagem alinha-se com a *Learning Analytics*, que visa melhorar a qualidade do ensino e da aprendizagem por meio da análise de dados sobre o comportamento dos estudantes (Huang *et al.*, 2020). Os *logs* de programação

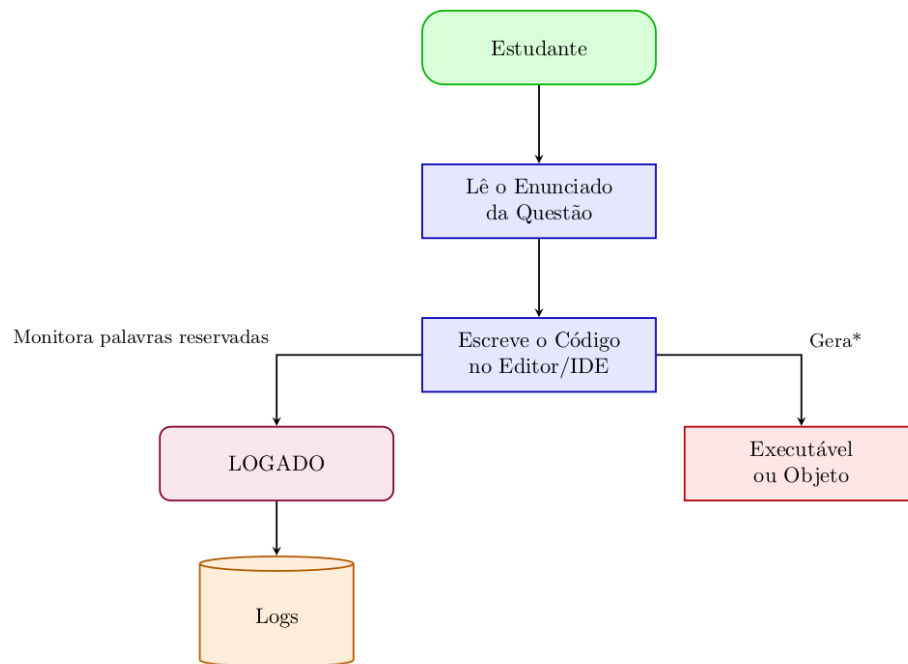


Figura 3 – Modelo de coleta de dados educacionais durante a atividade de codificação. O plugin LOGADO monitora o processo, gerando logs que registram a jornada do aprendiz, em contraste com o artefato final (código) que é o foco da avaliação tradicional.

funcionam, assim, como registros de aprendizagem (*learning records*), revelando a experiência pessoal e as reflexões do estudante durante o processo de desenvolvimento (Du; Wagner, 2005).

2.4 Design Science Research

Esta pesquisa adota a Design Science Research (DSR) como abordagem metodológica, amplamente utilizada em Computação para orientar o desenvolvimento de soluções tecnológicas voltadas a problemas práticos (Hevner *et al.*, 2004; Peffers *et al.*, 2007).

Diferentemente de pesquisas que apenas descrevem ou explicam fenômenos, a DSR concentra-se na criação de algo novo, um artefato, que atenda a uma necessidade real. No caso deste trabalho, o artefato é a extensão LOGADO para o Visual Studio Code.

Hevner *et al.* (Hevner *et al.*, 2004) estabelecem que pesquisas em DSR devem:

- produzir um artefato útil;
- endereçar um problema relevante;
- avaliar rigorosamente o que foi construído;
- comunicar claramente os resultados.

Peffers *et al.* (Peffers *et al.*, 2007) propõem um processo prático em seis etapas: identificar o problema, definir os objetivos da solução, projetar e desenvolver o artefato, demonstrar seu uso, avaliá-lo e comunicar os resultados.

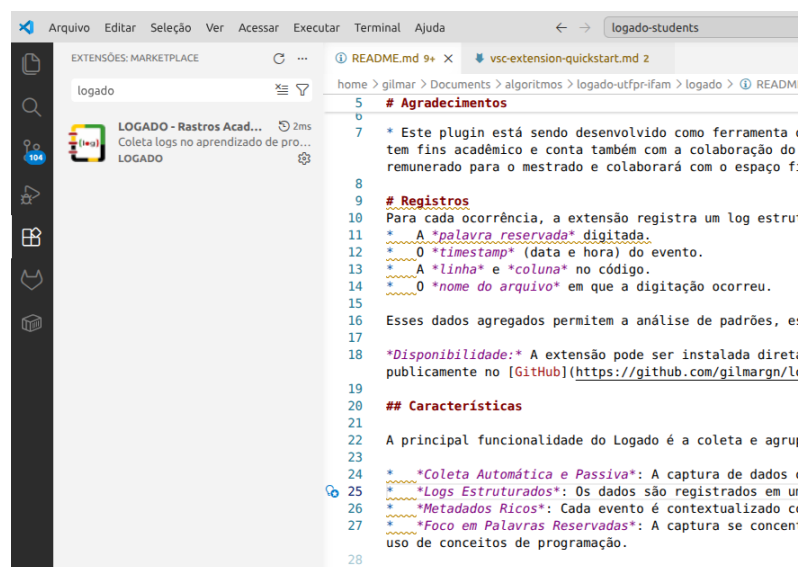
Este trabalho seguiu essas etapas. O problema identificado foi a falta de ferramentas de telemetria educacional no ensino de programação, para o qual se desenvolveu a extensão LOGADO. A avaliação ocorreu por meio da coleta e análise de logs gerados por estudantes em situação real de aula. Os resultados são apresentados e discutidos ao longo da dissertação, em especial nos Capítulos 4 e 5.

2.5 A Extensão LOGADO: Proposta de Implementação

A extensão LOGADO foi desenvolvida para coletar rastros de aprendizagem nas disciplinas de programação que usam JavaScript como linguagem e tem o Visual Studio Code (VS Code) como editor principal, que é amplamente adotado no contexto educacional por sua leveza, gratuidade e suporte a extensões. A ferramenta captura, de forma transparente e não intrusiva, as palavras reservadas digitadas pelos estudantes durante a resolução de problemas de programação, registrando eventos de edição e gerando arquivos de log no formato JSON.

2.5.1 Estado atual da implementação

Atualmente, a extensão LOGADO encontra-se na versão 0.0.5 e está disponível para instalação no Visual Studio Code por meio do arquivo `.vsix`, acessível através da paleta de comandos da IDE. O código-fonte completo, a documentação e as instruções de instalação estão disponíveis publicamente no repositório GitHub do projeto².



A extensão opera em segundo plano, não requerendo configuração adicional. Uma vez instalada, a coleta de logs registra as palavras reservadas utilizadas, os respectivos *timestamps*

² <https://github.com/gilmargn/logado-utfpr-ifam>

e a localização no código (arquivo, linha e coluna). O código é disponibilizado sob licença *MIT*³, em consonância com os princípios da ciência aberta.

2.5.2 Funcionalidades Principais

A extensão LOGADO foi projetada para ser uma ferramenta leve e integrada ao fluxo natural do estudante, operando de forma transparente, mas nunca oculta.

- **Transparência:** o estudante sabe exatamente quais informações são coletadas (palavras-chave, timestamp, arquivo, linha e coluna), não havendo coleta oculta de dados. Os logs são salvos em arquivos `.json` visíveis no explorador do VS Code.
- **Gerenciamento de privacidade:** a extensão opera em modo estritamente local. Os dados não são transmitidos para servidores externos sem consentimento explícito.
- **Exportação de dados:** os logs em JSON são salvos concomitantemente aos arquivos `.js`, podendo ser exportados para repositórios e compartilhados com pesquisadores ou instrutores.

2.5.3 Arquitetura e Armazenamento de Dados

A extensão foi desenvolvida em TypeScript, utilizando a API oficial do Visual Studio Code, com base na documentação oficial e em exemplos disponibilizados pela Microsoft. O trabalho consistiu em adaptar e aplicar os recursos existentes para a finalidade específica de coleta de logs educacionais.

O diferencial desta pesquisa não reside no código da extensão em si, mas na abordagem (telemetria de palavras reservadas como ferramenta pedagógica) e na aplicação (coleta de logs em contexto real de sala de aula). Os eventos capturados são salvos no formato JSON e armazenados localmente em arquivos de log, contendo palavra reservada, *timestamp*, arquivo, linha e coluna.

A extensão não realiza análises automáticas sobre os dados coletados. Sua função limita-se à captura e ao armazenamento estruturado dos rastros, cabendo a pesquisas futuras aprofundar a análise desses registros.

2.5.4 Notificação de Ativação

Ao iniciar o VS Code com a extensão LOGADO instalada, uma notificação visual é exibida no canto inferior direito da tela (Figura 6), informando ao estudante que os logs estão sendo coletados, em respeito aos princípios éticos de consentimento informado.

³ Licença permissiva que permite reuso, modificação e distribuição, desde que mantido o aviso de direitos autorais.

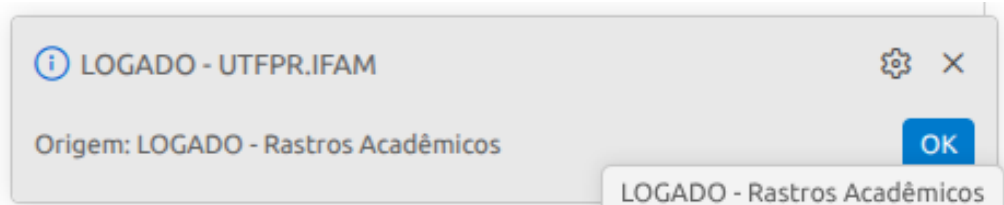


Figura 6 – Notificação exibida pela extensão LOGADO ao ser ativada

2.5.5 Geração de Logs

Para cada arquivo JavaScript editado, a extensão gera automaticamente um arquivo de log homônimo no formato JSON (.json), salvo no mesmo diretório do código-fonte. A Figura 7 ilustra essa correspondência.

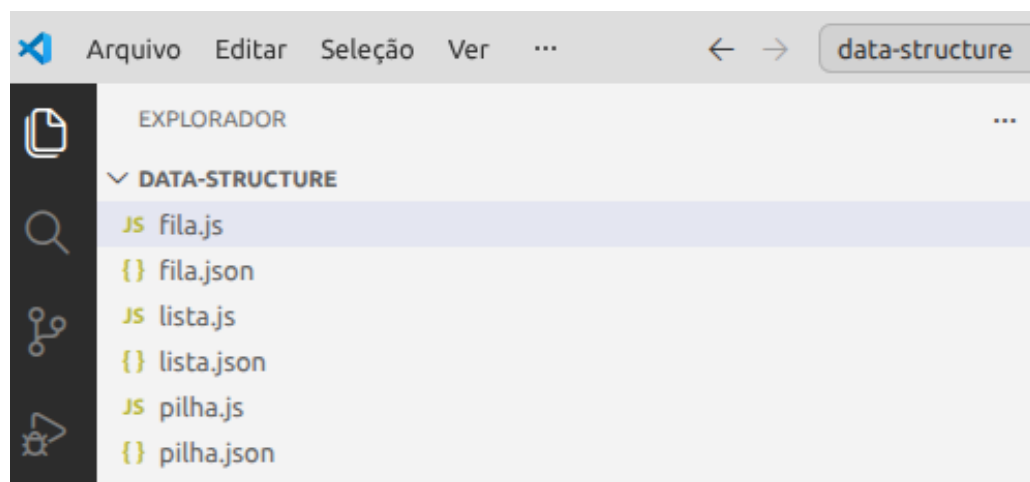


Figura 7 – Arquivos de log (.json) gerados para cada arquivo JavaScript

No exemplo apresentado, é possível observar a estrutura adotada durante as atividades da disciplina de Estrutura de Dados: os arquivos `lista.js`, `fila.js` e `pilha.js` representam as implementações solicitadas, enquanto os arquivos `lista.json`, `fila.json` e `pilha.json` armazenam os logs de palavras reservadas gerados durante a digitação de cada algoritmo. Essa separação preserva o código original do estudante e mantém os dados organizados, facilitando etapas posteriores de pré-processamento e agrupamento.

Para ilustrar a aplicação prática, considere o desenvolvimento do algoritmo de cálculo de impressão apresentado na Listagem 2.1. Durante sua implementação, a extensão LOGADO captura o uso de elementos da linguagem, gerando os registros estruturados apresentados na Listagem 2.2.

```

1 let numero_paginas = parseInt(
  prompt("Número de páginas"))
2 let tipo_impressao = prompt("Tipo
  de Impressão\n0 - Frente e
  verso\n1 - Frente")
3
4 let valor = 0
5
6 if(tipo_impressao=='0'){
7   valor = numero_paginas * 0.10
8   document.writeln(`Páginas
  impressas: ${numero_paginas}<
  br> Valor final: R$ ${valor.
  toFixed(2)} `)
9 }
10 else{
11   valor = numero_paginas * 0.15
12   document.writeln(`Páginas
  impressas: ${numero_paginas}<
  br> Valor final: R$ ${valor.
  toFixed(2)} `)
13 }
14

```

Listing 2.1 – Algoritmo de cálculo de impressão - exemplo didático

```

1 [
2   {
3     "keyword": "let",
4     "timestamp": "2026-02-24
  19:21:45",
5     "file": "impressao.js",
6     "line": 1,
7     "column": 1
8   },
9   {
10    "keyword": "parseInt",
11    "timestamp": "2026-02-24
  19:21:47",
12    "file": "impressao.js",
13    "line": 1,
14    "column": 18
15  },
16  {
17    "keyword": "prompt",
18    "timestamp": "2026-02-24
  19:21:48",
19    "file": "impressao.js",
20    "line": 1,
21    "column": 28
22  }
23 ]
24

```

Listing 2.2 – Exemplo de saída da extensão LOGADO

3 METODOLOGIA

Este capítulo descreve a abordagem metodológica adotada, composta por três elementos principais: uma Revisão Sistemática da Literatura (detalhada no Apêndice A), um *survey* exploratório (cujos resultados completos estão no Apêndice B) e a Design Science Research (DSR) para o desenvolvimento do artefato proposto.

A análise dos logs coletados assumiu caráter exploratório (Gil, 2018), uma vez que não partiu de categorias pré-definidas, mas buscou identificar padrões emergentes de uso de palavras reservadas, lacuna evidenciada pela Revisão Sistemática. O processo de agrupamento foi conduzido de forma não supervisionada, com documentação sistemática das decisões analíticas para garantir transparência e reprodutibilidade. Esta abordagem está alinhada aos princípios da Descoberta de Conhecimento em Bases de Dados (KDD) (Fayyad; Piatetsky-shapiro; Smyth, 1996) e à área de Mineração de Dados Educacionais (Baker; Yacef, 2009).

3.1 Abordagem Metodológica

Este trabalho foi conduzido no Instituto Federal de Educação, Ciência e Tecnologia do Amazonas, Campus Avançado Boca do Acre, utilizando uma abordagem metodológica mista. Foram combinados métodos de pesquisa bibliográfica e empírica para fundamentar o trabalho tanto no estado da arte quanto no contexto prático do problema investigado. Os procedimentos metodológicos da pesquisa respeitaram os preceitos éticos que regem pesquisas com seres humanos.

3.1.1 Revisão Sistemática da Literatura

Para mapear o estado da arte sobre o uso de logs no ensino de programação, foi conduzida uma Revisão Sistemática da Literatura (RSL) seguindo as diretrizes atualizadas para revisões sistemáticas em Engenharia de Software, incluindo o guia SEGRESS (Kitchenham; Madeyski; Budgen, 2023) e o protocolo PRISMA 2020 (Page *et al.*, 2021).

3.1.2 Survey Exploratório

Complementarmente à RSL, foi realizado um *survey* com 14 participantes (docentes, mestrandos em computação e pesquisadores da área de ensino de programação) com o objetivo de investigar o uso de ferramentas de captura de logs, construção de avaliações quantitativas e desenvolvimento de *datasets* acadêmicos. O instrumento de coleta continha 10 questões abordando logs, uso de ferramentas e IDEs. As respostas completas do *survey* e a análise detalhada encontram-se no **Apêndice B**.

3.1.3 Design Science Research

Este trabalho adotou a abordagem de Design Science Research (DSR), conforme os paradigmas fundacionais propostos por (Hevner *et al.*, 2004) e (Peppers *et al.*, 2007). Mais recentemente, (Hevner *et al.*, 2024) destacam que a transparência deve ser endereçada ao longo de todo o projeto de DSR, desde o planejamento até a disseminação dos resultados, alinhando-se aos princípios de design aberto e ciência aberta.

A presente pesquisa incorpora esses princípios de três formas: (i) o plugin LOGADO é disponibilizado publicamente no GitHub sob licença MIT; (ii) os logs coletados são visíveis e acessíveis ao estudante; (iii) as decisões de projeto e análise são documentadas de forma sistemática nesta dissertação.

O processo de DSR foi executado em quatro atividades principais:

3.1.3.1 Atividade 1: Identificação do Problema e Motivação

O problema central identificado foi a carência de ferramentas específicas para captura e análise de logs acadêmicos no ensino de programação. Essa lacuna é particularmente relevante no contexto da região Amazônica, onde instituições como o Instituto Federal do Amazonas (IFAM) vêm sendo implantadas por meio de políticas públicas de interiorização do ensino. A criação de campi avançados, como o de Boca do Acre, representa um esforço de levar educação profissional e tecnológica a regiões historicamente desassistidas. Nesse cenário, a produção de pesquisa aplicada e o desenvolvimento de soluções tecnológicas locais são formas de demonstrar os resultados concretos dessas políticas, mostrando que é possível gerar conhecimento de qualidade também fora dos grandes centros urbanos. A falta de ferramentas adequadas para coleta e análise de logs, no entanto, limita a capacidade dessas instituições de identificar dificuldades de aprendizagem de forma objetiva e baseada em dados.

3.1.3.2 Atividade 2: Definição dos Objetivos da Solução

Com base no problema identificado e no contexto institucional da pesquisa (um campus avançado do IFAM na região Amazônica), definiram-se os seguintes objetivos:

Objetivo Geral: Desenvolver e validar um plugin para coleta e análise de logs durante o processo de ensino-aprendizagem de programação, demonstrando que é possível produzir soluções tecnológicas de qualidade também em contextos de interiorização do ensino.

Objetivos Específicos:

1. Projetar e implementar um plugin de captura de eventos integrado à IDE adotada na disciplina.
2. Estabelecer um procedimento de coleta, sincronização e armazenamento dos dados gerados.

3. Estruturar os logs coletados em um dataset organizado e documentado.
4. Avaliar a viabilidade da abordagem por meio da coleta de dados em contexto real de sala de aula.

3.1.3.3 Atividade 3: Design e Desenvolvimento

O design e desenvolvimento seguiu uma abordagem iterativa, com ajustes contínuos durante as atividades práticas da disciplina. A arquitetura do artefato compreende quatro módulos principais:

- Módulo de captura: coleta palavras reservadas e timestamps.
- Módulo de estruturação: organiza os eventos em JSON.
- Módulo de armazenamento: gerencia os arquivos localmente.
- Módulo de configuração: permite verificar se a extensão está ativa e coletando logs corretamente.

3.1.3.4 Especificação de Requisitos do Artefato

Para garantir que o *plugin* LOGADO atendesse aos objetivos da pesquisa e às necessidades dos usuários finais, foi realizada uma etapa de especificação de requisitos. A análise inicial identificou dois atores principais, cujas necessidades são sumarizadas na Tabela 2.

Tabela 2 – Análise de atores e necessidades do sistema

Ação	Discente	Pesquisador
Funções necessárias	Enviar dados de execução	Desenvolver/testar plugin
Necessidade principal	Instalar e usar o plugin	Validar processamento de logs
Operações CRUD	Enviar (criar) logs	Gerenciar usuários e dataset

Com base nesta análise, foram elaborados diagramas de casos de uso para especificar as funcionalidades do sistema. A Figura 8 ilustra as interações do ator Discente, enquanto a Figura 9 detalha as funcionalidades para o Pesquisador.

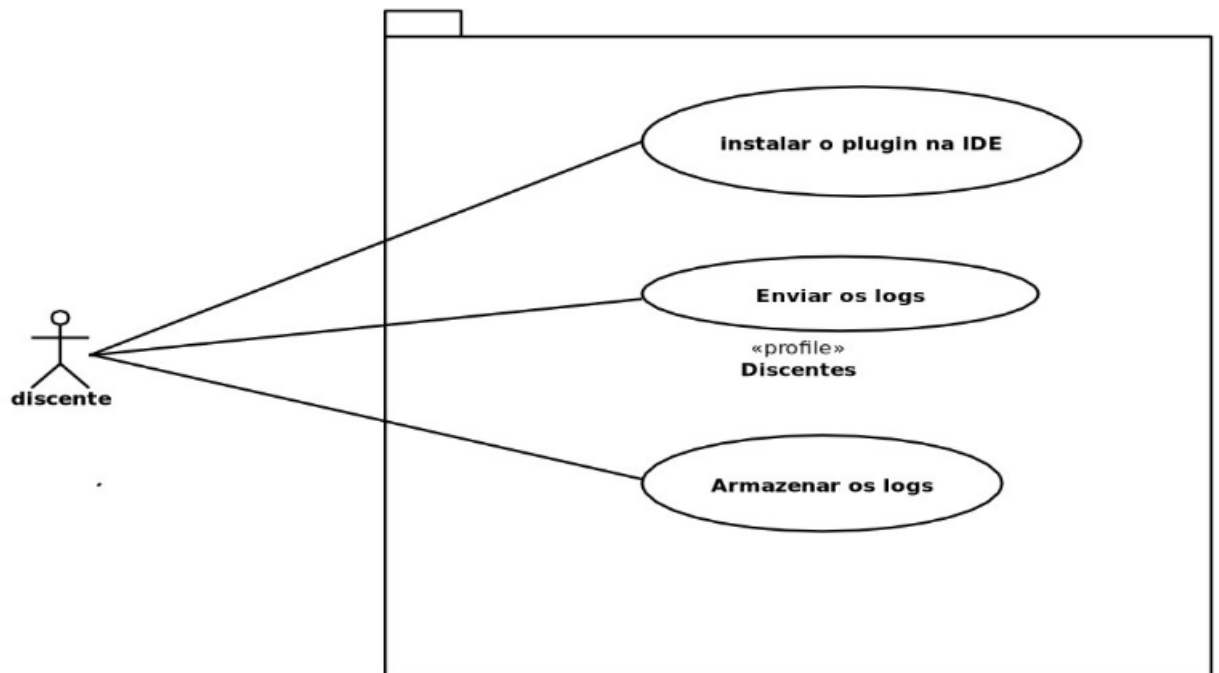


Figura 8 – Casos de uso do *plugin* LOGADO - Visão do Discente

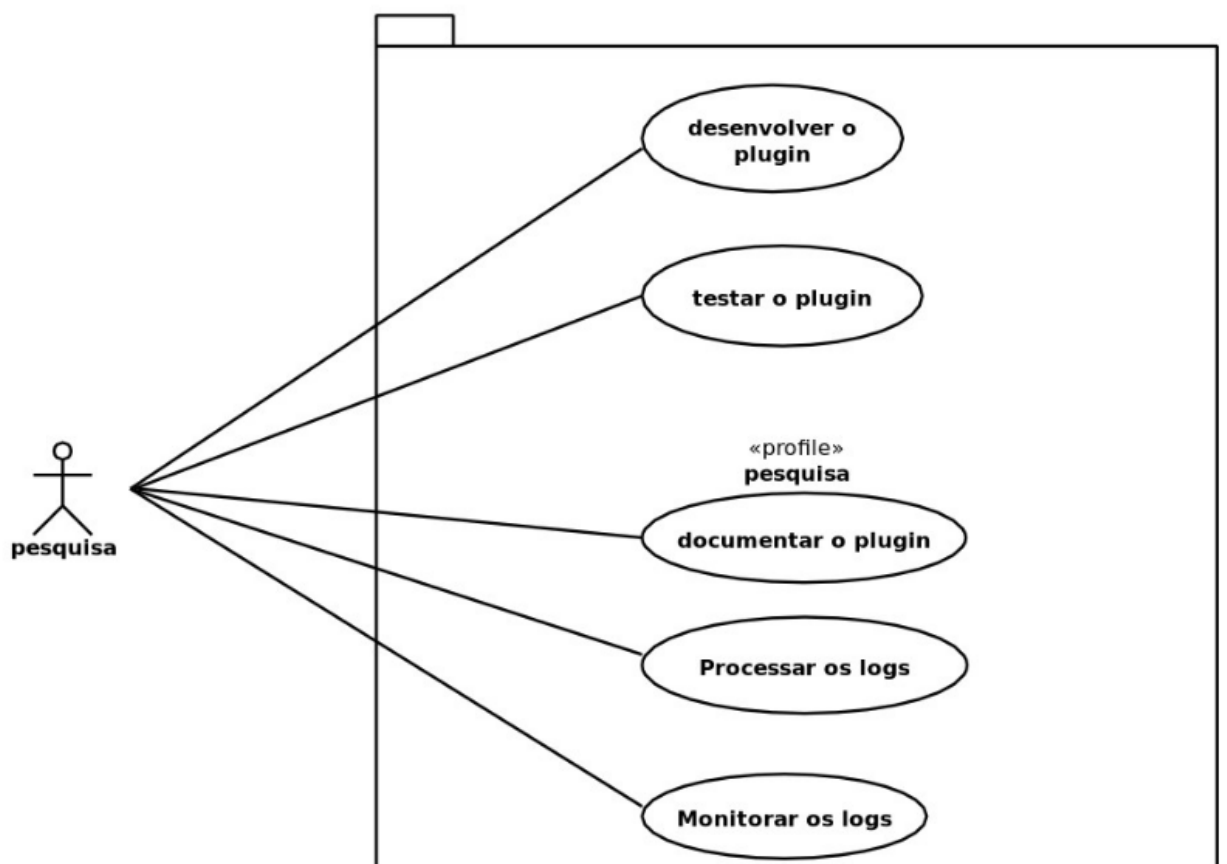


Figura 9 – Casos de uso do *plugin* LOGADO - Visão do Pesquisador

A partir dos casos de uso, foram derivados os requisitos formais do sistema:

3.1.3.4.1 Requisitos Funcionais

Os requisitos funcionais definem as funcionalidades que o sistema deve executar. Para o ator **Discente**, o *plugin* deve:

- **RF01:** Registrar automaticamente eventos de edição de código (ex.: inserção, deleção de caracteres).
- **RF02:** Capturar eventos de compilação e execução de código, incluindo sucesso ou falha.
- **RF03:** Armazenar localmente os logs com *timestamp* preciso.
- **RF04:** Transmitir os logs anonimizados para um servidor remoto de forma assíncrona.
- **RF05:** Exibir notificações de confirmação de operações para o usuário.

Para o ator **Pesquisador**, o *plugin* deve:

- **RF06:** Fornecer uma interface de configuração para parametrizar os eventos a serem capturados.
- **RF07:** Gerar identificadores únicos anônimos para cada sessão de uso.

3.1.3.4.2 Requisitos Não-Funcionais

Estes requisitos definem os critérios de qualidade para o sistema:

- **RNF01 (Desempenho):** O *plugin* não deve impactar significativamente o desempenho da IDE. O tempo de resposta deve ser inferior a 100ms para operações de registro.
- **RNF02 (Usabilidade):** A instalação e o uso devem ser simples, não requerendo configuração complexa por parte do discente.
- **RNF03 (Confiabilidade):** O *plugin* deve garantir a persistência dos logs mesmo em caso de fechamento inesperado da IDE.
- **RNF04 (Segurança e Privacidade):** Todos os dados pessoais devem ser anonimizados antes do envio ao servidor, conforme a LGPD.

3.1.3.4.3 Desenvolvimento do Artefato

Foi desenvolvido um *plugin* chamado LOGADO para o Visual Studio Code, com funcionalidades de captura de eventos de programação.

3.1.3.5 Atividade 4: Demonstração e Avaliação

3.1.3.5.1 *Contexto e Participantes*

A pesquisa foi realizada no IFAM Campus Avançado Boca do Acre. Participaram 23 estudantes do curso técnico em Informática, de um total de 25 que assinaram o Termo de Consentimento Livre e Esclarecido (TCLE).

3.1.3.5.2 *Procedimentos de Coleta*

- Duração: 5 semanas de aulas práticas.
- Local: Laboratório de Informática do campus.
- Ferramenta: Visual Studio Code com a extensão LOGADO instalada.
- Aspectos éticos: Aprovação do Comitê de Ética e aplicação do TCLE.

3.1.3.5.3 *Avaliação do Artefato*

A avaliação da extensão LOGADO foi conduzida em duas frentes: (i) verificação da integridade e completude dos logs gerados durante as atividades; (ii) análise da percepção dos estudantes sobre a transparência e o impacto da ferramenta em seu processo de aprendizagem, coletada por meio de um survey ao final do período de coleta (Apêndice B).

3.2 **Plano de Análise de Dados**

Os dados coletados foram analisados nas seguintes etapas:

3.2.1 **Pré-processamento e Limpeza**

Os estudantes participantes, após assinatura do Termo de Consentimento Livre e Esclarecido (TCLE), tiveram a opção de submeter voluntariamente seus códigos e os arquivos JSON gerados pela extensão LOGADO a um repositório público no GitHub. Para automatizar o processo de coleta desses dados, foi desenvolvido um script que:

- acessa o repositório e baixa os arquivos submetidos;
- organiza os logs em diretórios separados por estudante;
- anonimiza os dados, substituindo identificadores pessoais por códigos numéricos, em conformidade com a LGPD.

Após o download e a anonimização, os logs brutos foram submetidos a um processo de filtragem para remoção de entradas duplicadas ou inconsistentes.

3.2.2 Análise Exploratória e Identificação de Padrões

A análise dos logs coletados assumiu caráter exploratório (Gil, 2018), uma vez que não partiu de categorias pré-definidas, mas buscou identificar padrões emergentes de uso de palavras reservadas, lacuna evidenciada pela Revisão Sistemática (Seção ??).

Os dados gerados por este trabalho constituem uma base que pode, em pesquisas futuras, ser submetida a técnicas da Descoberta de Conhecimento em Bases de Dados (KDD) (Fayyad; Piatetsky-shapiro; Smyth, 1996) e da Mineração de Dados Educacionais (Baker; Yacef, 2009). Neste estudo, contudo, o foco recaiu sobre a análise exploratória descritiva, identificando padrões a partir da observação direta dos logs.

3.2.3 Potencial do Dataset e Possibilidades de Análise

O dataset gerado a partir da coleta de logs abre possibilidades para investigações futuras sobre padrões de aprendizagem em programação. Com base nos dados coletados, é possível vislumbrar análises como:

- sequências de declaração de variáveis (ex.: múltiplos `let` consecutivos);
- transições conceituais (ex.: mudança de `let` para `var` em diferentes escopos);
- sincronia temporal entre estudantes, possivelmente refletindo dinâmicas de mediação pedagógica;
- associações entre palavras-chave e desempenho acadêmico.

Não é objetivo deste trabalho esgotar essas análises, mas sim disponibilizar os dados para que outros pesquisadores possam realizá-las. O foco da presente pesquisa recai sobre o desenvolvimento do plugin LOGADO, a experiência de coleta no IFAM Campus Avançado Boca do Acre e a participação voluntária de estudantes de nível técnico em uma pesquisa de mestrado, demonstrando que é possível produzir ciência e tecnologia de qualidade também em contextos de interiorização do ensino.

3.3 Percurso de Execução da Pesquisa

As atividades previstas no cronograma da pesquisa foram executadas ao longo do período de mestrado, com os seguintes marcos alcançados:

- **Desenvolvimento do plugin:** extensão LOGADO implementada na versão 0.0.5, disponível para instalação via arquivo `.vsix`;

- **Coleta de dados:** realizada com 25 estudantes que assinaram o TCLE, com participação efetiva de 23 ao longo de 5 semanas, totalizando 19 repositórios;
- **Publicação associada:** uma versão preliminar do trabalho foi aceita para apresentação na LACLO (Conferência Latinoamericana de Tecnologias de Aprendizagem). O capítulo completo foi posteriormente publicado em livro pela Springer, disponível em: <https://link.springer.com/book/10.1007/978-981-95-7580-0>.

A pesquisa respeitou os preceitos éticos da Resolução n° 510 de 2016.

4 RESULTADOS E DISCUSSÕES

Este capítulo apresenta os resultados obtidos a partir da coleta de logs de palavras reservadas utilizando o LOGADO. Conforme descrito na metodologia (Capítulo 3), a análise assume caráter exploratório, buscando identificar padrões emergentes de uso de palavras-chave da linguagem de programação JavaScript sem categorias pré-definidas.

4.1 Caracterização da amostra

Foram coletados logs de aprendizado de estudantes do curso técnico em Informática do IFAM Campus Boca do Acre, durante atividades práticas da disciplina de Estrutura de Dados. Participaram efetivamente 23 estudantes ao longo de 5 semanas, totalizando 19 repositórios registrados. O número de estudantes é diferente do número de repositórios, pois alguns estudantes não quiseram participar da pesquisa em algum momento ou assinaram o Termo de Consentimento Livre e Esclarecido (TCLE) e não compareceram às aulas.

Processamento dos dados. Os logs coletados foram processados utilizando a linguagem Python versão 3.x, com o auxílio das bibliotecas Pandas (Mckinney, 2011) para manipulação e agregação dos dados, Matplotlib (Hunter, 2007) e Seaborn (Waskom, 2021) para a geração das visualizações, e NetworkX (Hagberg; Schult; Swart, 2008) para a construção da rede de co-ocorrência entre palavras-chave.

A Figura 10 apresenta as 20 palavras-chave mais frequentes nos logs coletados.

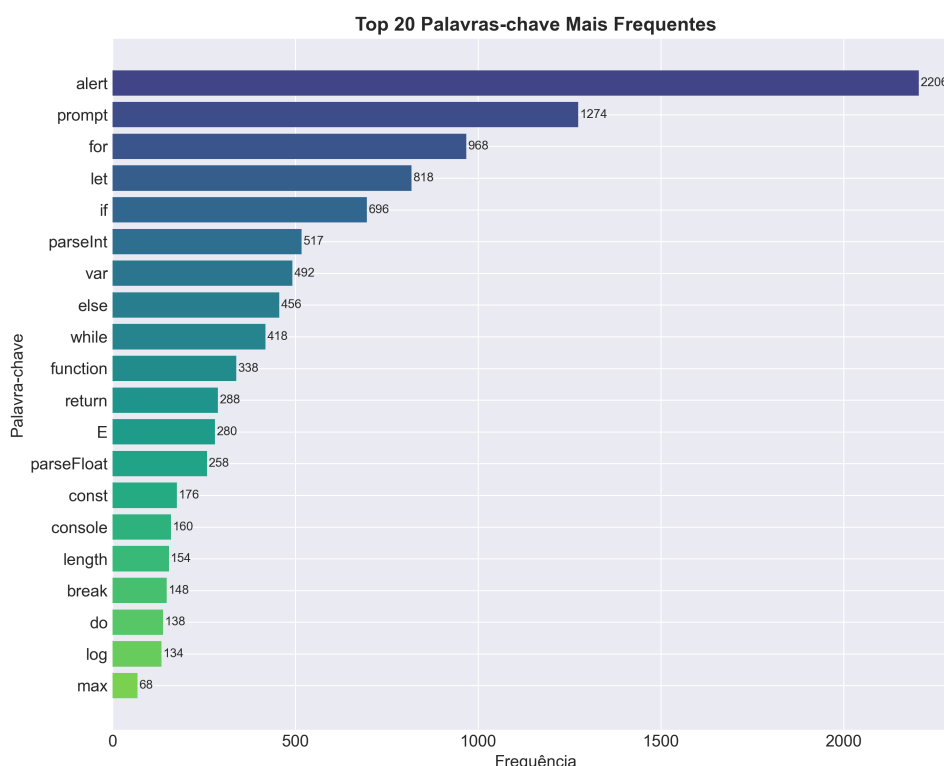


Figura 10 – Top 20 palavras-chave mais frequentes nos logs coletados

Fonte: Elaborado pelo autor.

Observa-se que `alert`, `prompt` e `for` estão entre as palavras-chave mais frequentes nos logs coletados. Este resultado reflete a natureza das atividades práticas da disciplina de Estrutura de Dados, nas quais os algoritmos implementados demandam: (i) interação com o usuário para entrada de dados (`prompt`); (ii) exibição de resultados (`alert`); e (iii) estruturas de repetição (`for`) para processamento de dados.

A palavra `let`, com 818 ocorrências (quarta posição), indica a necessidade de declaração de variáveis para armazenamento e manipulação de dados. Sua frequência, embora expressiva, é inferior à das palavras relacionadas à interação e repetição, sugerindo que, no contexto observado, a ênfase das atividades recaiu mais sobre o fluxo de entrada-processamento-saída do que sobre a complexidade das estruturas de dados em si.

Vale notar a presença relativamente modesta de `console` (160 ocorrências), indicando que a depuração por meio do console não foi uma estratégia frequente entre os estudantes — possivelmente refletindo baixa autonomia na verificação do código ou preferência por métodos mais diretos de exibição de resultados como `alert`.

Este resultado deve ser interpretado à luz do contexto institucional: trata-se de um curso técnico em Informática na forma subsequente, com estudantes que possuem diferentes níveis de experiência prévia em programação. Aliado a isso, o calendário acadêmico e a necessidade de reprodução de códigos apresentados em sala de aula influenciam diretamente os padrões observados. Os rastros capturados evidenciam, portanto, a tentativa dos estudantes de reproduzir exemplos da docência e de aplicar palavras-chave consultadas na documentação da linguagem.

Vale notar que `var` aparece com menor frequência que `let`, o que pode indicar a adoção das práticas modernas de JavaScript ensinadas em aula, embora transições entre os dois sejam observadas em situações envolvendo escopo de variáveis.

A Figura 11 mostra como os logs se distribuem entre os diferentes arquivos criados pelos estudantes.

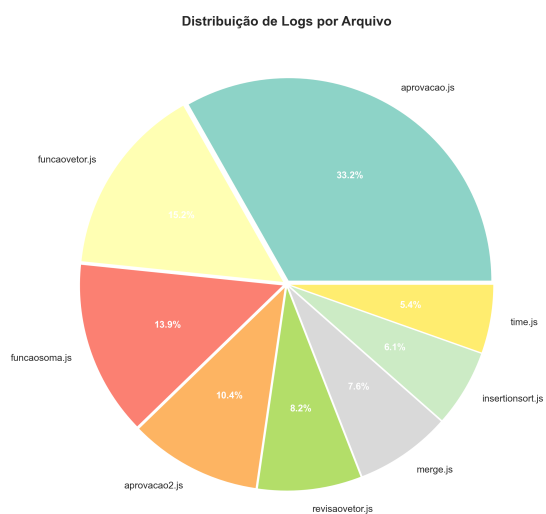


Figura 11 – Distribuição dos logs por arquivo

Fonte: Elaborado pelo autor.

Nota-se que poucos arquivos concentram a maior parte dos logs, enquanto a maioria dos arquivos apresenta baixa frequência de registros. Este comportamento era esperado: algoritmos mais complexos exigem maior quantidade de palavras-chave e lógica mais elaborada, gerando naturalmente mais rastros. Além disso, a padronização de nomenclatura adotada pela equipe permitiu identificar quais arquivos correspondiam aos exercícios principais da disciplina e quais eram secundários ou experimentais.

A Figura 12 mostra a distribuição temporal dos logs ao longo do dia.

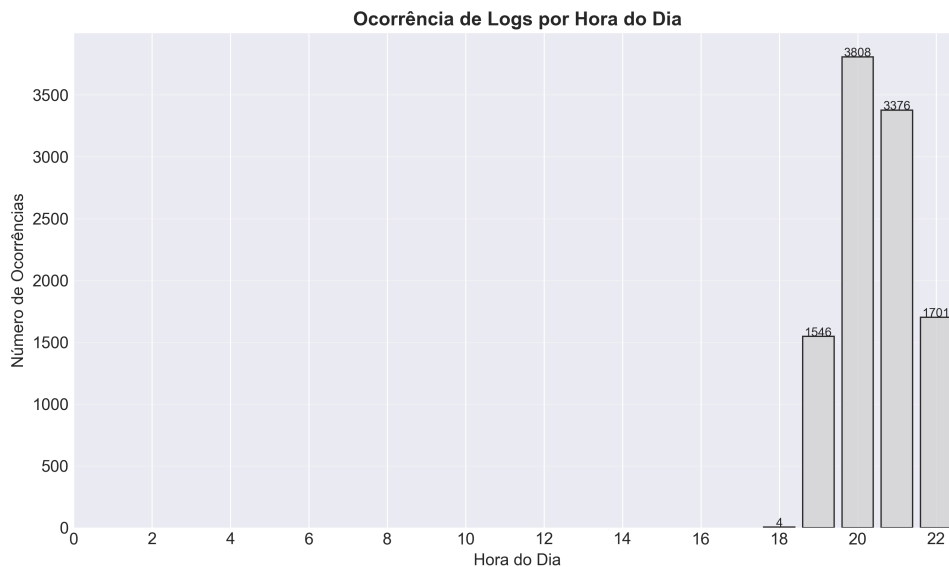


Figura 12 – Ocorrência de logs por hora do dia

Fonte: Elaborado pelo autor.

Observa-se que os logs concentram-se em períodos específicos, com picos acentuados em determinados horários. Este padrão reflete a dinâmica da sala de aula: o professor resolve os algoritmos no quadro em momentos definidos e, a partir daí, os estudantes reproduzem as soluções em seus editores. As ocorrências em horários similares indicam baixa autonomia na produção dos códigos, uma vez que poucos estudantes iniciam ou desenvolvem as atividades por conta própria antes da mediação docente.

As variações observadas entre os horários podem ser atribuídas a diferenças individuais, como ritmo de digitação ou tempo necessário para compreender e transcrever a solução apresentada. Contudo, a ausência de registros significativos fora dos períodos de exposição do professor sugere dependência da mediação pedagógica para o avanço nas atividades.

A Figura 13 apresenta a rede de co-ocorrência entre as palavras-chave utilizadas pelos estudantes.

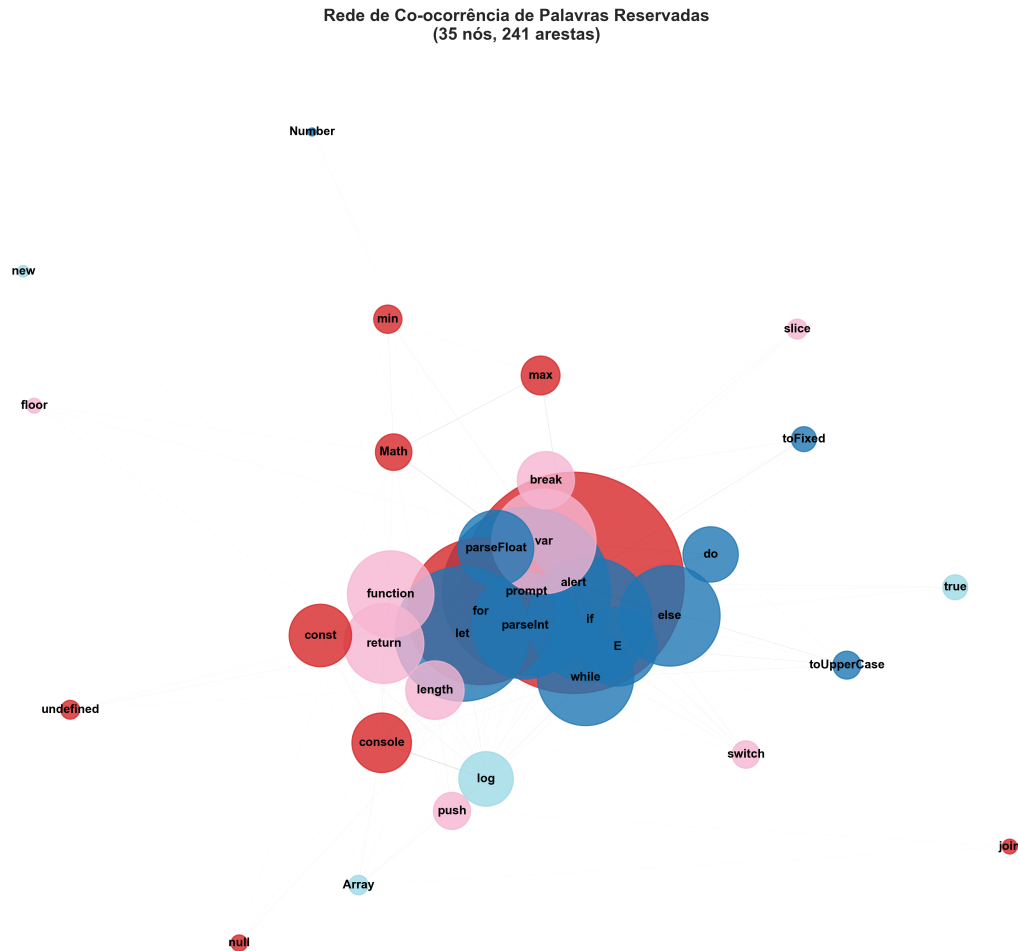


Figura 13 – Rede de co-ocorrência entre palavras reservadas

Fonte: Elaborado pelo autor.

A rede revela que os estudantes utilizam um conjunto relativamente restrito de palavras-chave para resolver os algoritmos propostos, evidenciando repertório limitado ou fortemente influenciado pelos exemplos apresentados em sala de aula. Observa-se a formação de clusters bem definidos: condicionais (`if`, `else`), laços de repetição (`for`, `while`), interação com o usuário (`prompt`, `alert`) e funções (`function`, `return`).

Um aspecto relevante na estrutura da rede é o papel da palavra `return`, que atua como elemento de conexão entre o cluster de funções e o cluster de condicionais. Esta interconexão reflete a prática comum em algoritmos que utilizam estruturas condicionais no interior de funções para controlar o fluxo de execução e retornar valores conforme a lógica implementada.

Destaca-se também a presença de `let` e `var` como palavras-chave para declaração de variáveis. Embora ambas cumpram funções semelhantes, a análise contextual dos logs — aliada ao conhecimento da dinâmica da turma — indica que `let` foi predominantemente utilizado para declarações de variáveis locais (dentro de blocos e laços), enquanto `var` apareceu em situações que exigiam escopo global ou funções específicas de contexto. Esta distinção, embora sutil na rede, é pedagogicamente significativa: revela a transição conceitual dos estudantes em relação ao entendimento de escopo de variáveis.

Não foram identificadas palavras isoladas sem conexões na rede, o que sugere que todo o vocabulário utilizado pelos estudantes está integrado às estruturas básicas da linguagem necessárias para resolver os algoritmos propostos. Este padrão, contudo, pode refletir tanto a adequação das atividades quanto a limitação do repertório dos discentes.

A Figura 14 apresenta as palavras-chave com maior centralidade na rede de co-ocorrência.

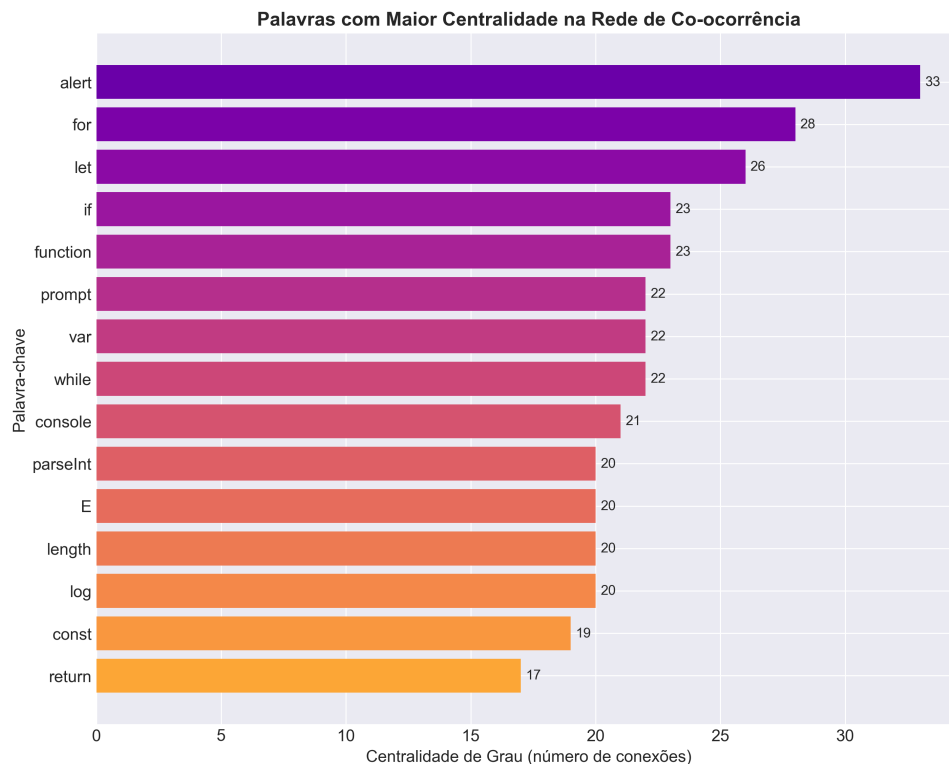


Figura 14 – Palavras com maior centralidade na rede de co-ocorrência

Fonte: Elaborado pelo autor.

Das três formas de exibição de resultados — `document.write`, `console.log` e `alert` — os estudantes preferiram `alert` pela simplicidade e foco no resultado.

Entre os laços de repetição, `for` foi mais utilizado que `while`. Esta diferença explica-se pela dificuldade dos estudantes com `while`: frequentemente esquecem de iterar a variável de controle, gerando laços infinitos. O `for`, por sua vez, quando apresenta erro de sintaxe, simplesmente não executa, comportamento mais fácil de diagnosticar.

Quanto à declaração de variáveis, `let` foi o mais frequente entre `const`, `var` e `let`, indicando preferência pela forma moderna.

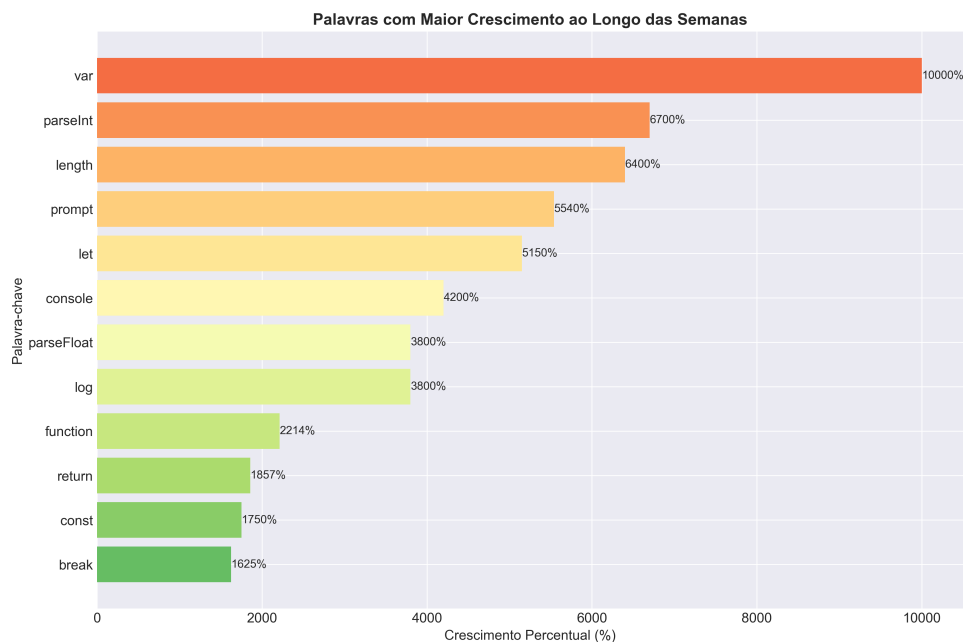
A Tabela 3 sintetiza as diferenças observadas entre as duas estruturas de repetição no contexto de aprendizagem.

Tabela 3 – Comparação entre `for` e `while` no contexto observado

Aspecto	<code>for</code>	<code>while</code>
Frequência de uso	Alta	Baixa
Comportamento com erro de sintaxe	Não executa (seguro)	Pode executar indefinidamente
Erro comum	Esquecer a condição de parada	Esquecer a iteração da variável
Consequência do erro	Código não produz saída	Laço infinito (travamento)
Facilidade de diagnóstico	Alta	Baixa

Fonte: Elaborado pelo autor.

A Figura 15 apresenta as palavras-chave que tiveram maior crescimento de frequência ao longo das semanas.

**Figura 15 – Palavras com maior crescimento ao longo das semanas**

Fonte: Elaborado pelo autor.

O resultado revela que não houve ampliação significativa do vocabulário técnico dos estudantes durante o período analisado. As palavras emergentes pertencem a categorias já dominadas desde o início: declaração de variáveis (`let`), conversão de tipos (`cast`), operações de medição (`length`) e interação com o usuário (`prompt`, `console`).

Do ponto de vista pedagógico, a ausência de evolução do tesouro técnico é um achado relevante. Esperar-se-ia que, ao longo de disciplinas como Estrutura de Dados, palavras como `array`, `object`, `class`, `return` ou `try` surgissem entre as emergentes. Sua ausência sugere que:

- Os estudantes não incorporaram novos conceitos ao seu repertório ativo;
- O vocabulário utilizado manteve-se restrito ao mínimo necessário para reproduzir os exemplos do professor;

- A dependência pedagógica, identificada na análise temporal, também se manifesta na estagnação do vocabulário técnico.

Este padrão corrobora a hipótese de que a mediação intensiva do professor, embora necessária no contexto, pode limitar a autonomia e a ampliação do repertório dos estudantes quando não acompanhada de estímulos à exploração independente da linguagem.

Este resultado reforça o argumento central desta pesquisa: os logs de palavras reservadas revelam dimensões do processo de aprendizagem que não seriam capturadas pela análise do código final. A estagnação do vocabulário técnico — evidenciada pela ausência de palavras emergentes mais avançadas — é um fenômeno que apenas os rastros de construção, analisados longitudinalmente, podem revelar.

4.1.1 Estrutura do dataset gerado

Após o pré-processamento e a consolidação dos logs coletados, foi gerado um *dataset* com **4.614 registros**, organizados nas seguintes colunas:

- `arquivo.json`: nome do arquivo de log original (ex.: `vaquinha.json`, `jogos.json`);
- `keyword`: palavra reservada capturada (ex.: `prompt`, `for`, `let`, `alert`);
- `timestamp`: data e hora do evento no formato `YYYY-MM-DD HH:MM:SS`;
- `arquivo.js`: nome do arquivo fonte onde a palavra foi digitada;
- `linha`: número da linha no arquivo fonte;
- `coluna`: posição da coluna onde a palavra foi digitada;
- `data_processamento`: data de consolidação do registro no formato ISO.

A Tabela 4 apresenta uma amostra real dos registros coletados, ilustrando a diversidade de palavras-chave e arquivos envolvidos nas atividades.

Tabela 4 – Amostra real do dataset gerado

arquivo.json	keyword	timestamp	arquivo.js	linha	coluna	data_processamento
vaquinha.json	prompt	2026-03-12 19:48:00	vaquinha.js	3	29	2026-04-01 08:13:07
vaquinha.json	for	2026-03-12 19:49:18	vaquinha.js	4	5	2026-04-01 08:13:07
vaquinha.json	alert	2026-03-12 19:56:13	vaquinha.js	8	5	2026-04-01 08:13:07
jogos.json	for	2026-02-26 20:04:05	jogos.js	3	1	2026-04-01 08:13:07
jogos.json	let	2026-02-26 20:04:05	jogos.js	3	6	2026-04-01 08:13:07
par_impar.json	prompt	2026-02-26 22:20:19	par_impar.js	2	22	2026-04-01 08:13:07
par_impar.json	else	2026-02-26 22:24:30	par_impar.js	8	1	2026-04-01 08:13:07
funcao_soma.json	function	2026-03-25 21:17:02	funcao_soma.js	1	1	2026-04-01 08:13:07
funcao_soma.json	return	2026-03-25 21:17:54	funcao_soma.js	2	5	2026-04-01 08:13:07
funcao_soma.json	prompt	2026-03-25 21:18:47	funcao_soma.js	4	16	2026-04-01 08:13:07

Os dados apresentados na Tabela 4 ilustram a variedade de contextos de programação capturados: desde algoritmos simples como verificação de par/ímpar (`par_impar.js`) e cálculos com estruturas de repetição (`vaquinha.js`), até implementações de funções (`funcao_soma.js`) e jogos (`jogos.js`). Essa diversidade demonstra a aplicabilidade da extensão LOGADO em diferentes tipos de atividades práticas da disciplina de Estrutura de Dados.

4.1.2 Contribuições para a telemetria educacional

O agrupamento de logs fortalece a telemetria educacional ao oferecer evidências objetivas sobre dimensões do processo de aprendizagem que os critérios tradicionais de avaliação não capturam. Estes critérios concentram-se predominantemente na tríade léxica, semântica e lógica dos programas, ou em métricas computacionais de execução — como número de linhas, funções, métodos e outros indicadores típicos da engenharia de software (Pressman; Maxim, 2021; Sommerville, 2019). Contudo, tais abordagens não consideram os **rastros da construção do código** ao longo do tempo: a sequência de decisões sintáticas, as tentativas de correção, a alternância entre declarações e interações, e outros padrões processuais que evidenciam o *como* o estudante construiu sua solução, não apenas o produto final ou sua conformidade com critérios estáticos.

4.1.3 Limitações contextuais: a contenção de estudantes

Um fenômeno adicional que caracteriza o contexto investigado, e que não poderia deixar de ser registrado, é a chamada *contenção de estudantes*. Diante de limitações estruturais (falta de computadores, formação prévia fragilizada) e institucionais (pressão por retenção e evasão), docentes raramente aplicam os critérios de avaliação que seriam esperados em disciplinas de algoritmos e estrutura de dados. Esta flexibilização, embora compreensível no curto prazo para manter estudantes matriculados, reforça a descrença na necessidade do conhecimento técnico e compromete a qualidade da formação.

Este trabalho não propõe, nem poderia propor, uma solução para a contenção — um problema estrutural que escapa ao escopo da pesquisa em telemetria educacional. Reconhecer sua existência é, contudo, essencial para situar adequadamente os resultados e as limitações do estudo. A contribuição que se oferece é modesta, mas não trivial: fornecer ao professor evidências baseadas em dados que, mesmo em um contexto de contenção, permitem decisões pedagógicas mais informadas e intervenções mais precisas.

4.1.4 Comportamento observado: interação dos estudantes com os logs

Durante o período de coleta de dados, observou-se um fenômeno não previsto inicialmente: os estudantes frequentemente acessavam e examinavam os arquivos `.json` gerados pela extensão LOGADO, mesmo sem compreender completamente sua finalidade ou o formato dos dados.

Este comportamento, documentado por meio da observação direta em laboratório e dos próprios registros de acesso aos arquivos, revela aspectos importantes:

- **Efeito da transparência:** a presença visível dos arquivos de log no explorador do VS Code despertou a curiosidade dos estudantes, confirmando que a coleta de dados não passava despercebida — um aspecto ético relevante;
- **Estranhamento tecnológico:** o fato de estudantes examinarem arquivos cujo propósito não compreendiam plenamente sugere uma lacuna na compreensão sobre telemetria e registro de dados, possivelmente associada à falta de familiaridade com práticas de *logging*;
- **Oportunidade pedagógica:** este comportamento abre espaço para discussões em sala de aula sobre transparência de dados, privacidade e rastreamento digital — temas transversais relevantes na formação técnica.

Embora a análise aprofundada deste fenômeno escape ao escopo central deste trabalho — focado no agrupamento de logs para identificação de padrões de aprendizagem —, sua ocorrência é documentada como um achado secundário que sugere direções para pesquisas futuras sobre a percepção discente acerca de ferramentas de telemetria educacional.

5 CONSIDERAÇÕES FINAIS

A presente pesquisa, embora tenha alcançado seus objetivos e gerado contribuições originais para a área de telemetria educacional no ensino de programação, apresenta limitações que precisam ser explicitamente reconhecidas. Tais limitações, longe de invalidarem os resultados obtidos, qualificam o escopo das conclusões e apontam direções para a continuidade da investigação.

5.1 Limitações do estudo

5.1.1 Especificidade do contexto investigado

Os dados foram coletados em uma única instituição de ensino, com estudantes de nível técnico cursando Estrutura de Dados, caracterizados por: (i) acesso limitado ou inexistente a computadores fora do ambiente escolar; (ii) baixa autonomia na elaboração de algoritmos, com forte dependência de mediação docente; (iii) descrença na utilidade do conhecimento técnico, alimentada pelo histórico de egressos fora do mercado de trabalho e pelo uso generalizado de LLMs; e (iv) um contexto institucional de contenção, no qual critérios de avaliação são frequentemente flexibilizados para evitar evasão. Os padrões de uso de palavras reservadas identificados refletem, portanto, este conjunto específico de condições, não sendo diretamente generalizáveis para outros contextos educacionais.

5.1.2 Plugin não testado em outras instituições

O artefato desenvolvido (plugin para VS Code) foi utilizado exclusivamente na turma investigada. Não há, neste momento, evidências sobre seu comportamento, usabilidade ou eficácia em outros ambientes institucionais, com diferentes perfis de estudantes, abordagens pedagógicas ou infraestrutura disponível.

5.1.3 Natureza exploratória e ausência de categorias prévias

Conforme explicitado na metodologia, adotou-se uma abordagem exploratória de descoberta de padrões, sem categorias analíticas pré-definidas. Esta escolha, embora legítima e adequada ao caráter inovador da investigação, introduz um grau de subjetividade no processo de identificação e nomeação dos padrões emergentes. A replicabilidade dos achados, portanto, depende da transparência com que as decisões analíticas foram documentadas.

5.1.4 Alcance limitado da contribuição

O trabalho não propõe, nem teria condições de propor, soluções para problemas estruturais do ensino de programação — como a falta de infraestrutura, a desvalorização docente, as políticas institucionais de contenção ou o impacto das LLMs na percepção de valor do conhecimento técnico. A contribuição situa-se em um nível mais modesto, porém não trivial: oferecer ao professor evidências baseadas em dados para decisões pedagógicas mais informadas dentro das margens de manobra que seu contexto permite.

5.1.5 Trabalhos Futuros

As limitações acima apontam diretamente para desdobramentos possíveis e necessários desta pesquisa:

Replicação em outras instituições e contextos. A aplicação do plugin em diferentes instituições de ensino — com níveis de formação variados (superior, técnico integrado, cursos livres), abordagens pedagógicas distintas (metodologias ativas, sala de aula invertida, aprendizagem baseada em projetos) e diferentes condições de infraestrutura — permitiria: (i) validar a ferramenta em novos cenários; (ii) comparar padrões de uso de palavras reservadas entre contextos; e (iii) construir uma tipologia mais robusta de padrões associados a diferentes perfis de estudantes e práticas docentes.

Estudos em contextos de maior autonomia. Particularmente relevante seria replicar a pesquisa em ambientes nos quais os estudantes elaboram algoritmos sem mediação direta do professor (por exemplo, em atividades extraclasse, em avaliações individuais ou em projetos de longo prazo). A comparação entre os padrões observados neste estudo (marcados pela sincronia temporal e pela cópia do quadro) e aqueles obtidos em contextos de maior autonomia poderia revelar diferenças significativas no processo de construção de conhecimento.

Análise longitudinal. Acompanhar uma mesma turma ao longo de diferentes disciplinas ou semestres permitiria investigar como os padrões de uso de palavras reservadas evoluem com o tempo, possivelmente refletindo o desenvolvimento de competências mais sofisticadas de estruturação algorítmica.

Integração com outras fontes de dados. Trabalhos futuros podem combinar os logs de palavras reservadas com outras fontes de evidência — como entrevistas com estudantes, questionários de percepção de aprendizagem, registros de desempenho acadêmico ou logs de interação com LLMs — para uma compreensão mais rica e multidimensional do fenômeno investigado.

Desenvolvimento de dashboards pedagógicos. A partir da experiência acumulada, um desdobramento natural é o desenvolvimento de interfaces de visualização (*dashboards*) que apresentem os padrões agrupados de forma intuitiva para o professor, reduzindo a necessidade de conhecimento técnico em análise de dados e ampliando a adoção da telemetria educacional na prática docente cotidiana.

Investigação da relação entre padrões de logs e intervenções pedagógicas. Estudos futuros podem manipular deliberadamente aspectos da prática docente (por exemplo, antecipar explicações sobre escopo quando determinados padrões são identificados) e medir o impacto dessas intervenções nos padrões subsequentes de uso de palavras reservadas, estabelecendo relações causais entre telemetria e melhoria da aprendizagem.

Comparação entre código do docente e dos discentes. Estudos futuros podem analisar a similaridade entre o código apresentado pelo professor em sala de aula e os códigos produzidos pelos estudantes, considerando métricas como tempo de execução, uso de palavras-chave, quantidade de linhas e estrutura algorítmica. Essa comparação permitiria avaliar, de forma individualizada, o grau de compreensão e autonomia de cada estudante na reprodução e adaptação dos exemplos fornecidos.

REFERÊNCIAS

- ANNAMAA, A. Thonny, a python ide for learning programming. *In: PROCEEDINGS OF THE 2015 ACM CONFERENCE ON INNOVATION AND TECHNOLOGY IN COMPUTER SCIENCE EDUCATION*. 2015. **Anais [...]** [S.l.: s.n.], 2015. p. 343–343.
- ARABYARMOHAMADY, S.; MORADI, H.; ASADPOUR, M. A coding style-based plagiarism detection. *In: IEEE. PROCEEDINGS OF 2012 INTERNATIONAL CONFERENCE ON INTERACTIVE MOBILE AND COMPUTER AIDED LEARNING (IMCL)*. 2012. **Anais [...]** [S.l.], 2012. p. 180–186.
- BAKER, R. S.; YACEF, K. The state of educational data mining in 2009: A review and future visions. **Journal of educational data mining**, v. 1, n. 1, p. 3–17, 2009.
- BECKER, K. *et al.* Besouro: A framework for exploring compliance rules in automatic tdd behavior assessment. **Information and Software Technology**, Elsevier v. 57,, p. 494–508, 2015.
- CLEMENTE, R. G. **Uma arquitetura para processamento de eventos de log em tempo real**. 2008. 15–77 p. Dissertação (Mestrado) — Pontifícia Universidade Católica do Rio de Janeiro 2008.
- DU, H. S.; WAGNER, C. Learning with weblogs: An empirical investigation. *In: IEEE. PROCEEDINGS OF THE 38TH ANNUAL HAWAII INTERNATIONAL CONFERENCE ON SYSTEM SCIENCES*. 2005. **Anais [...]** [S.l.], 2005. p. 7b–7b.
- FAYYAD, U.; PIATETSKY-SHAPIO, G.; SMYTH, P. From data mining to knowledge discovery in databases. **AI magazine**, American Association for Artificial Intelligence v. 17, n. 3, p. 37–37, 1996.
- FELISBINO, C. M.; NETO, A. G. S. S.; BASTOS, L. C. Supporting to the teaching and learning process in object orientation during the construction of class diagrams. *In: PROCEEDINGS OF THE XXXII BRAZILIAN SYMPOSIUM ON SOFTWARE ENGINEERING*. 2018. **Anais [...]** [S.l.: s.n.], 2018. p. 338–347.
- FU, Q. *et al.* Where do developers log? an empirical study on logging practices in industry. *In: COMPANION PROCEEDINGS OF THE 36TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING*. 2014, Hyderabad India. **Anais [...]** Hyderabad India: ACM, 2014. p. 24–33. ISBN 978-1-4503-2768-8. Disponível em: <https://dl.acm.org/doi/10.1145/2591062.2591175>.
- GATEV, R. Observability: Logs, metrics, and traces. *In: Introducing Distributed Application Runtime (Dapr): Simplifying Microservices Applications Development Through Proven and Reusable Patterns and Practices*. Berkeley, CA: Apress, 2021. p. 233–252. ISBN 978-1-4842-6998-5. Disponível em: https://doi.org/10.1007/978-1-4842-6998-5_12.
- GIL, A. C. **Métodos e técnicas de pesquisa social**. 7. ed. São Paulo: Atlas, 2018.
- GU, S. *et al.* Logging practices in software engineering: A systematic mapping study. **IEEE Transactions on Software Engineering**, IEEE v. 49, n. 2, p. 902–923, 2022.
- GUPTA, A.; JINDAL, M.; GOYAL, A. Identification of student programming patterns through clickstream data. *In: IEEE. 2024 IEEE INTERNATIONAL CONFERENCE ON COMPUTING, POWER AND COMMUNICATION TECHNOLOGIES (IC2PCT)*. 5., 2024. **Anais [...]** [S.l.], 2024. p. 1153–1158.

HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. Exploring network structure, dynamics, and function using networkx. *In*: VAROQUAUX, G.; VAUGHT, T.; MILLMAN, J. (Ed.). PROCEEDINGS OF THE 7TH PYTHON IN SCIENCE CONFERENCE (SCIPY2008). 2008, Pasadena, CA USA. **Anais [...]** Pasadena, CA USA: [s.n.], 2008. p. 11–15.

HEVNER, A. R. *et al.* Design science in information systems research. **MIS Quarterly**, JSTOR v. 28, n. 1, p. 75–105, 2004.

HEVNER, A. R. *et al.* Transparency in design science research. **Decision Support Systems**, Elsevier v. 182, p. 114236, 2024.

HUANG, A. Y. *et al.* Predicting students' academic performance by using educational big data and learning analytics: evaluation of classification methods and learning logs. **Interactive Learning Environments**, Taylor & Francis v. 28, n. 2, p. 206–230, 2020.

HUNTER, J. D. Matplotlib: A 2d graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

IFAM. **Projeto Pedagógico do Curso Técnico de Nível Médio em Informática, na Forma Subsequente**. [S.l.], 2020. Disponível em: <https://portal.ifam.edu.br/documents/754/INFORMATICA.pdf>.

KENNEDY, A. R. **Towards a data-driven analysis of programming tutorials' telemetry to improve the educational experience in introductory programming courses**. 2015. Tese (Doutorado) 2015.

KITCHENHAM, B.; MADEYSKI, L.; BUDGEN, D. Segress: Software engineering guidelines for reporting secondary studies. **IEEE Transactions on Software Engineering**, v. 49, n. 3, p. 1273–1298, 2023.

LIMA, N. A. *et al.* **Avaliação da aprendizagem nos Institutos Federais em Goiás durante a pandemia: uma investigação documental**. [S.l.], 2022.

LIU, C. *et al.* User behavior discovery from low-level software execution log. **IEEE Transactions on Electrical and Electronic Engineering**, Wiley Online Library v. 13, n. 11, p. 1624–1632, 2018.

MANGAROSKA, K. *et al.* Exploring students' cognitive and affective states during problem solving through multimodal data: Lessons learned from a programming activity. **Computer Assisted Learning**, v. 38, n. 1, p. 40–59, fev. 2022. ISSN 0266-4909, 1365-2729. Disponível em: <https://onlinelibrary.wiley.com/doi/10.1111/jcal.12590>.

MCKINNEY, W. pandas: a foundational python library for data analysis and statistics. **Python for High Performance and Scientific Computing**, v. 14, n. 9, p. 1–9, 2011.

NGUYEN, B.-A.; HA, T.-V. T.; CHEN, H.-M. Analyzing git log in an code-quality aware automated programming assessment system: A case study. **TVU Journal of Science**, v. 13, n. 3, p. 1–15, 2023. ISSN 2815-6099. Disponível em: <https://journal.tvu.edu.vn/index.php/journal/article/view/2430>.

OVERFLOW, S. **2025 Stack Overflow Developer Survey**. [S.l.], 2025. Disponível em: <https://survey.stackoverflow.co/2025/technology>.

PAGE, M. J. *et al.* The prisma 2020 statement: an updated guideline for reporting systematic reviews. **Systematic reviews**, Springer v. 10, n. 1, p. 1–11, 2021.

PEFFERS, K. *et al.* A design science research methodology for information systems research. **Journal of management information systems**, Taylor & Francis v. 24, n. 3, p. 45–77, 2007.

PEREIRA, F. D. *et al.* Using learning analytics in the amazonas: understanding students' behaviour in introductory programming. **British journal of educational technology**, Wiley Online Library v. 51, n. 4, p. 955–972, 2020.

PRESSMAN, R. S. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. [S.l.]: AMGH, 2020.

PRESSMAN, R. S.; MAXIM, B. R. **Software Engineering: A Practitioner's Approach**. 9th. ed. New York: McGraw-Hill, 2021.

QIAN, Y.; LEHMAN, J. Students' misconceptions and other difficulties in introductory programming: A literature review. **ACM Transactions on Computing Education (TOCE)**, ACM New York, NY, USA v. 18, n. 1, p. 1–24, 2017.

RAABE, A. L. A.; SILVA, J. d. Um ambiente para atendimento as dificuldades de aprendizagem de algoritmos. *In*: SN. XIII WORKSHOP DE EDUCAÇÃO EM COMPUTAÇÃO (WEI'2005). SÃO LEOPOLDO, RS, BRASIL. 3 n. 5., 2005. **Anais [...]** [S.l.], 2005.

RICE, R. E.; BORGMAN, C. L. The use of computer-monitored data in information science and communication research. **Journal of the American Society for Information Science**, Wiley Online Library v. 34, n. 4, p. 247–256, 1983.

SILVA, E. S. *et al.* Previsão de indicadores de dificuldade de questões de programação a partir de métricas do código de solução. *In*: SBC. ANAIS DO XXXIII SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO. 2022. **Anais [...]** [S.l.], 2022. p. 859–870.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. [S.l.]: Pearson, 2016.

SOMMERVILLE, I. **Engineering Software Products**. Boston: Pearson, 2019.

UMEZAWA, K. *et al.* Analysis of logic errors utilizing a large amount of file history during programming learning. *In*: IEEE. 2020 IEEE INTERNATIONAL CONFERENCE ON TEACHING, ASSESSMENT, AND LEARNING FOR ENGINEERING (TALE). 2020. **Anais [...]** [S.l.], 2020. p. 630–634.

VALENTE, M. T. **Engenharia de Software Moderna**. 1. ed. [S.l.]: Independente, 2020.

WASKOM, M. L. seaborn: statistical data visualization. **Journal of Open Source Software**, v. 6, n. 60, p. 3021, 2021.

YANG, N. *et al.* An interview study of how developers use execution logs in embedded software engineering. *In*: IEEE. 2021 IEEE/ACM 43RD INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP). 2021. **Anais [...]** [S.l.], 2021. p. 61–70.

APÊNDICE A – Mapeamento Sistemático da Literatura

A.1 Objetivo e questões de pesquisa

As pesquisas conduzidas nos repositórios de trabalhos, com base nas *strings* de busca apresentadas na Tabela 6, resultaram em 163 estudos após um processo de refinamento e testes iterativos das entradas. Esses trabalhos foram então categorizados e analisados em fases sequenciais, visando garantir uma revisão sistemática e abrangente.

A.2 Estratégia de busca e repositórios

A.2.1 Repositórios consultados

A pesquisa foi realizada em cinco bibliotecas virtuais, utilizando como critérios de busca palavras-chave e *strings* de busca com operadores lógicos *AND* e *OR*. Os repositórios de pesquisa utilizados são apresentados na Tabela 5.

Repositório	Resultados
Scopus	23
IEEE Xplore	2
SOL SBC	2
ACM	127
WILEY	9

Tabela 5 – Lista de repositórios de pesquisa utilizados no estudo.

A.2.2 Strings de busca

As combinações de palavras-chave e operadores utilizadas na pesquisa são apresentadas na Tabela 6.

Palavras-chave e Operadores
programming AND teaching AND log AND NOT (log-based) AND NOT (metrics) AND systematic AND review
programming AND teaching AND log AND NOT (log-based) AND (integrate AND development AND environment) AND NOT (visual AND programming) AND NOT (web AND application)
telemetry AND education AND programming AND log
ontology AND education AND programming AND log AND dataset
artefact AND cognitive AND learning
background AND teaching AND log AND NOT (log-based) AND (systematic AND review)
log AND ("teaching programming"OR "learning programming") AND ("IDE"OR "integrated development environment")) NOT ("log-based"OR "lms"OR "learning management system"
(log AND ("teaching programming"OR "learning programming") AND ("IDE"OR "integrated development environment")) NOT ("log-based"OR "lms"OR "learning management system"OR "game-based"OR "game based"OR "gamification")

Tabela 6 – Combinações de palavras-chave e operadores utilizados na pesquisa.

A.2.3 Idioma dos trabalhos

Os trabalhos foram selecionados em português e inglês. A inclusão do português justifica-se pela relevância de pesquisas nacionais, especialmente no contexto das Instituições de Ensino Federais brasileiras. O inglês, por sua vez, foi escolhido por ser o idioma predominante em publicações acadêmicas internacionais, como conferências e periódicos de alto impacto.

A.3 Critérios de inclusão e exclusão

A.3.1 Critérios de inclusão

Após a execução das buscas utilizando as *strings* definidas, os artigos identificados foram submetidos a um processo de triagem. Foram adotados os seguintes critérios para inclusão:

1. Relevância temática: O artigo deve abordar explicitamente o uso de logs no ensino de programação, conforme verificado pela análise de títulos e resumos.
2. Revisão sistemática: Trabalhos que se caracterizam como revisões sistemáticas da literatura também foram incluídos, devido à sua contribuição para a compreensão do estado da arte no tema.

A.3.2 Critérios de exclusão

Foram excluídos textos que, mesmo contendo a *string* de busca, retornaram resultados relacionados a:

1. *Log-based*: abordagens ou técnicas que utilizam logs como base principal (por exemplo, sistemas de monitoramento ou análise de logs).
2. Uso de logs em outras áreas da educação: aplicações de logs em contextos educacionais que não envolvem diretamente o ensino de programação.
3. *Learning Management Systems (LMS)*: trabalhos que abordam o uso de logs em plataformas de gestão de aprendizagem (LMS), como Moodle, Canvas ou Blackboard, sem foco específico no ensino de programação.
4. *Game-based learning*: trabalhos que utilizam jogos ou mecânicas de jogos como principal estratégia de ensino, sem foco no uso de logs para análise de aprendizagem.

A.4 Resultados da busca e seleção de estudos

O processo de seleção, ilustrado na Figura 16, resultou na inclusão de 17 estudos para análise qualitativa.

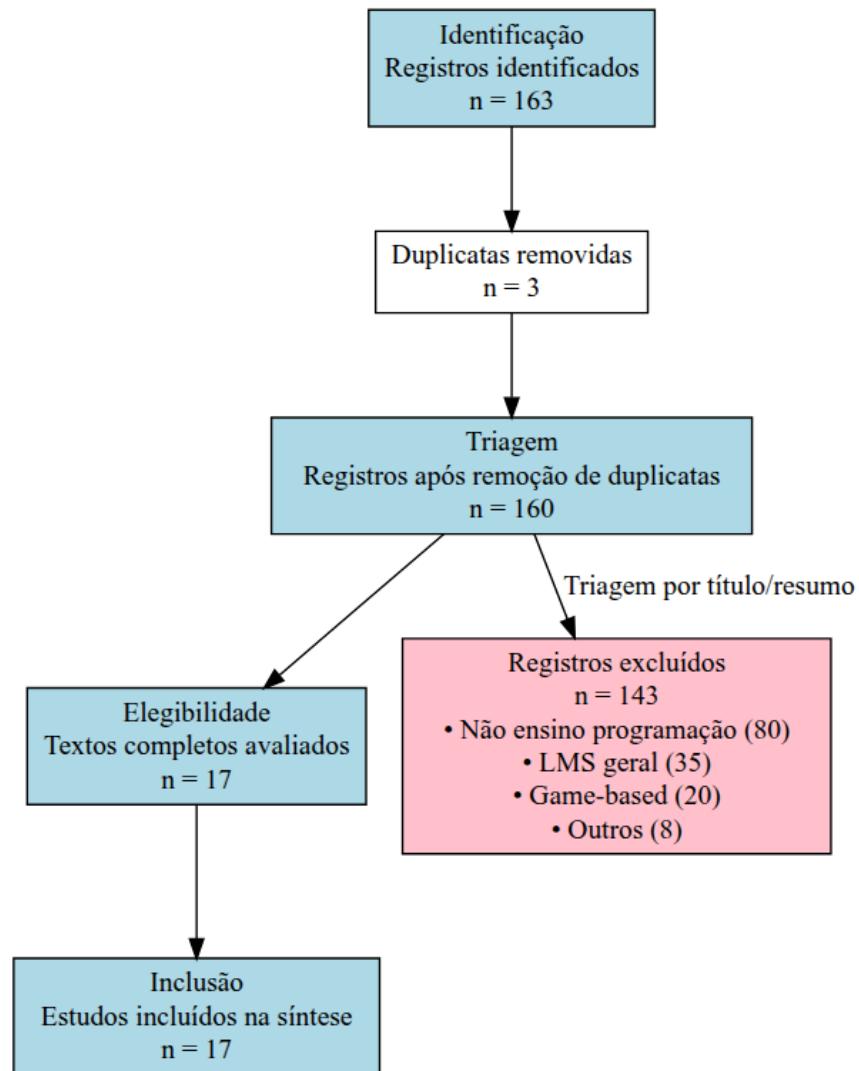


Figura 16 – Fluxograma PRISMA do processo de seleção de estudos

A.5 Caracterização dos estudos incluídos

A Tabela 7 sumariza as principais características dos 17 estudos selecionados.

Estudo	Ano	Foco Principal	Tipo de Log
Arabyarmohamady et al.	2012	Detecção de plágio	Estilo de codificação
Annamaa	2015	IDE educacional	Logs de sessão Python
Raabe et al.	2005	Dificuldades de aprendizagem	Ambiente educacional
Liu et al.	2018	Comportamento do usuário	Logs de execução
Umezawa et al.	2020	Análise de erros lógicos	Histórico de arquivos
Pereira et al.	2020	Comportamento estudantil	Logs de IDE + Online Judge
Gu et al.	2022	Práticas de logging	Revisão sistemática
Silva et al.	2022	Predição de dificuldade	Métricas de código
Mangaroska et al.	2022	Estados cognitivos	Logs Eclipse + multimodal
Da Silva	2022	Análise de comunidades	Comportamento online
Nguyen et al.	2023	Avaliação automática	Git logs + qualidade
Educomp	2023	Complexidade vs dificuldade	Métricas de exercícios
Gupta et al.	2024	Padrões de programação	Clickstream data
Arguson et al.	2022	Erros de compilação	Logs de compilação
Liz-Domínguez et al.	2022	Desempenho acadêmico	Logs de LMS
Yang et al.	2021	Uso de logs na indústria	Práticas profissionais
Cândido et al.	2021	Monitoramento baseado em logs	Revisão sistemática

Tabela 7 – Características dos 17 estudos incluídos na síntese qualitativa

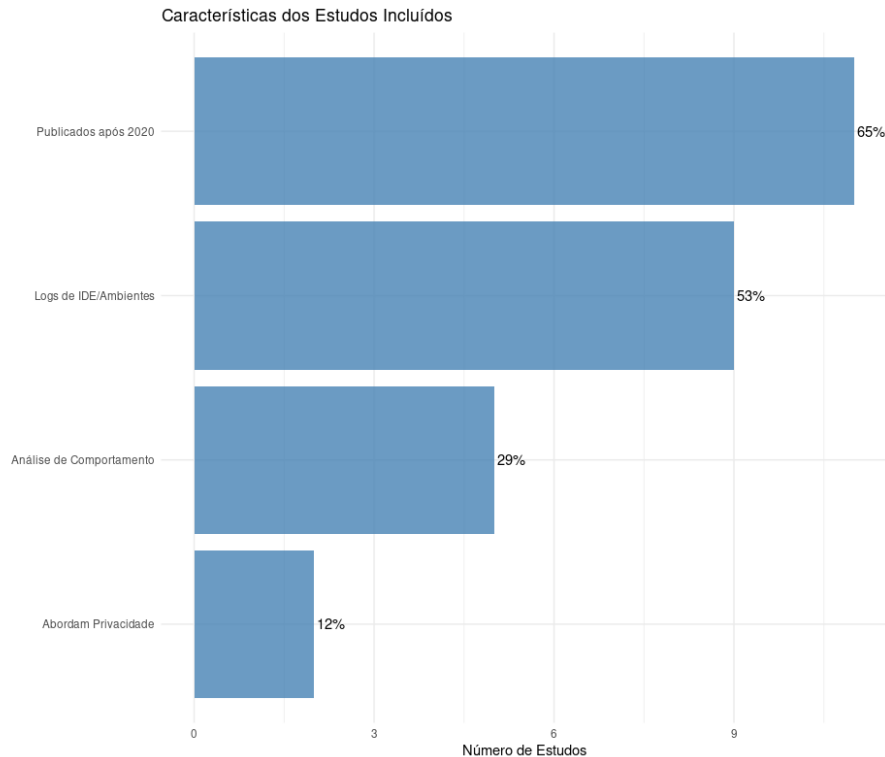


Figura 17 – Distribuição das características principais dos estudos incluídos (n=17)

A análise dos 17 estudos revela um interesse crescente na área, com 65% das publicações concentradas após 2020. A maioria dos estudos (53%) utiliza logs de IDEs ou ambientes específicos, enquanto apenas 12% abordam aspectos de privacidade e ética.

A.6 Respostas às questões de pesquisa

A.6.1 QP1: Quais são as principais aplicações do uso de logs no ensino de programação?

Os objetivos identificados foram categorizados em quatro áreas principais:

- **Deteccção de Dificuldades** (6 estudos): Umezawa (2020), Pereira (2020), Silva (2022), Educomp (2023), Arguson (2022), Raabe (2005)
- **Análise de Comportamento** (5 estudos): Gupta (2024), Liu (2018), Mangaroska (2022), Da Silva (2022), Liz-Domínguez (2022)
- **Qualidade de Código** (3 estudos): Arabyarmohamady (2012), Nguyen (2023), Annamaa (2015)
- **Práticas e Ferramentas** (3 estudos): Gu (2022), Yang (2021), Cândido (2021)

De acordo com (Gu *et al.*, 2022), "A obtenção de logs é um processo que deve ser guiado por quatro questões fundamentais, conhecidas como 3W1H:

- **Why** (Por quê): Qual é o objetivo da coleta de logs?

- Where (Onde): Em quais partes do sistema ou código os logs devem ser coletados?
- What (O que): Quais informações ou eventos devem ser registrados nos logs?
- How (Como): De que forma os logs serão coletados, armazenados e analisados?

Essas questões são essenciais para garantir que a geração de logs seja eficiente, relevante e alinhada aos objetivos do projeto, seja no desenvolvimento de software ou no contexto educacional. No ensino de programação, por exemplo, a aplicação do 3W1H pode ajudar a definir métricas e técnicas para monitorar o progresso dos estudantes e identificar dificuldades de aprendizado.

A abordagem feita por (Silva *et al.*, 2022) busca "prever indicadores de dificuldade de exercícios de programação a partir de métricas de complexidade do código dos instrutores, identificando registros de logs de JOs e relacionando esforço e dificuldade dos estudantes durante o desenvolvimento do código de solução".

De acordo com (Gupta; Jindal; Goyal, 2024), "os dados de cliques (clickstream) e logs de atividades dos estudantes em plataformas de programação podem ser processados para extrair informações relevantes, como padrões de programação e relações entre características do código e o desempenho dos alunos. Essa análise permite que os instrutores identifiquem previamente estudantes com dificuldades ou baixo desempenho, ajustando suas metodologias de ensino e oferecendo suporte direcionado. Além disso, ao acompanhar a evolução do estilo de codificação dos alunos, os educadores podem avaliar a eficácia de suas estratégias pedagógicas e garantir que nenhum estudante seja deixado para trás em um mundo tecnologicamente avançado".

O ensino de programação vai além da transmissão de conhecimentos técnicos, incluindo orientações sobre licenciamento de códigos e boas práticas de desenvolvimento. Nesse contexto, o combate ao plágio é uma preocupação relevante, e os logs surgem como uma ferramenta valiosa para apoiar a avaliação dos estudantes. Como argumenta (Arabyarmohamady; Moradi; Asadpour, 2012), características extraídas de cada código, como o estilo de programação, são armazenadas em um histórico (user profile). Esses dados permitem identificar mudanças no estilo de codificação e detectar se um código foi realmente desenvolvido pelo autor declarado ou se foi escrito por outra pessoa com um estilo diferente. Dessa forma, os logs não apenas auxiliam na identificação de plágio, mas também contribuem para uma avaliação mais precisa e personalizada do aprendizado dos estudantes.

Outra aplicação relevante para a geração de logs é a criação de *datasets*, como demonstrado no trabalho de (Umezawa *et al.*, 2020). O estudo detecta diversos elementos em códigos-fonte, como declarações de variáveis, funções, estruturas condicionais, laços de repetição, operações de leitura e escrita, espaçamento, pontuação, expressões lógicas e matemáticas, além de comentários. Esses dados são organizados em tabelas para análise. Um exemplo interessante é o uso do espaçamento: embora não afete a execução do código, padrões de espaçamento 'incorretos' ou idênticos podem indicar que vários estudantes estão cometendo o mesmo erro ou copiando trechos de código uns dos outros. Isso demonstra que os logs vão

além da identificação de erros de lógica ou sintaxe, podendo revelar padrões de comportamento e práticas comuns entre os estudantes. Além disso, o trabalho destaca a importância de estruturas de controle, como declarações 'for' e 'if', e a frequência de alterações em variáveis e números, que refletem a natureza repetitiva dos exercícios universitários. Esses insights mostram o potencial dos logs para apoiar educadores na identificação de dificuldades e na melhoria do ensino de programação.

Em síntese, as principais aplicações de logs no ensino de programação, identificadas nos 17 estudos analisados, concentram-se em quatro categorias: detecção de dificuldades (35% dos estudos), análise de comportamento (29%), qualidade de código (18%) e práticas/ferramentas (18%). Predominam abordagens voltadas à identificação precoce de estudantes com baixo desempenho e à análise de padrões de codificação.

A.6.2 QP2: Quais ferramentas, métodos e métricas são mais utilizados para capturar, armazenar e analisar logs em ambientes de ensino de programação?

No trabalho de (Pereira *et al.*, 2020), foi desenvolvida uma IDE para obter todas as informações. "O CodeBench fornece o seu próprio editor de código (IDE), concebido para ser simples e fácil de utilizar por principiantes. Todas as ações executadas pelos alunos no IDE (por exemplo, teclas digitadas, submissões, colagem de código, cliques do mouse, transições de separadores ou de janelas, etc.) são registradas e gravadas em arquivos de registro no lado do servidor."

Para (Liu *et al.*, 2018), "Cada evento representa o registro de uma chamada de método durante a execução do software. Portanto, todos os eventos podem ser alinhados com padrões de comportamento previamente identificados. Ao utilizar um log de execução de software e os padrões de comportamento treinados como entrada, realizamos a correspondência baseada em alinhamento (alignment-based matching) para obter um log abstrato de operações do usuário."Essa técnica pode ser aplicada no ensino de programação para analisar as interações dos estudantes com ferramentas de desenvolvimento, identificar erros comuns e fornecer feedback personalizado com base em seu comportamento durante a codificação.

O trabalho de (Mangaroska *et al.*, 2022) utiliza algumas ferramentas para capturar a interação do usuário, contudo, na questão de logs, foi utilizada uma extensão para o Eclipse e foi definida assim: "Log data: Um plugin para Eclipse que possui uma aba, visualização (Trætteberg *et al.*, 2016), foi utilizado para obter comportamentos de leitura e escrita dos estudantes. Este plugin armazena o estado do programa toda vez que o estudante salva o programa, usando o atalho ou botão de salvar."

A ferramenta desenvolvida por (Annamaa, 2015), uma IDE projetada na Universidade de Tartu com interface amigável para iniciantes em Python, coleta informações detalhadas sobre cada sessão de programação. Esses dados incluem eventos como carregar, salvar e modificar arquivos, além de diferenciar textos copiados de textos digitados. As informações coletadas permitem reproduzir todo o processo de construção do programa e as atividades realizadas

no shell. Para isso, a ferramenta oferece uma janela separada onde o usuário pode selecionar um arquivo de log e visualizar a reprodução dos eventos em velocidade ajustável. Essa funcionalidade é particularmente útil para educadores, pois permite analisar o processo de desenvolvimento dos estudantes e identificar possíveis dificuldades.

Nos últimos anos, a educação em programação tem passado por uma transformação significativa com a introdução de Sistemas Automatizados de Avaliação de Programação (APASs). Esses sistemas revolucionaram a forma como as tarefas de programação são avaliadas, oferecendo vantagens tanto para educadores quanto para estudantes. O ProgEdu, desenvolvido por (Nguyen; Ha; Chen, 2023), é um exemplo de aplicação baseada na web que combina as funcionalidades de um Sistema de Gestão de Aprendizagem (LMS) com um APAS. A ferramenta permite que os estudantes acessem atividades, submetam códigos e recebam feedback automatizado, facilitando a identificação de erros e o aprimoramento das habilidades de programação. Além disso, o ProgEdu é uma aplicação baseada em Docker, o que garante portabilidade e facilidade de implantação. A automação do processo de avaliação não apenas reduz a carga de trabalho dos instrutores, mas também oferece avaliações consistentes e imparciais, aplicando critérios predefinidos de forma objetiva. Essa abordagem é particularmente relevante no contexto da Educação a Distância (EAD), onde a popularização de serviços baseados na web tem se tornado cada vez mais comum.

Em síntese, as ferramentas mais frequentes para captura de logs são IDEs específicas (Code-Bench, Thonny), plugins para IDEs comerciais (Eclipse) e sistemas web (ProgEdu). Os métodos predominantes incluem registro de eventos de teclado, mouse, salvamento de arquivos e submissões. As métricas mais comuns são: número de compilações, taxa de erro por linha de código, tempo entre submissões, quantidade de tentativas até o acerto e padrões de espaçamento e indentação.

A.7 Lacunas identificadas e trabalhos futuros

A análise revelou lacunas significativas:

- **Padronização:** Ausência de modelos comuns para logs educacionais
- **Privacidade:** Poucos estudos abordam aspectos éticos
- **Contexto Brasileiro:** Limitada pesquisa em instituições brasileiras

APÊNDICE B – Survey

Para entender o uso de logs no ensino de programação, buscamos informações de outras instituições de ensino. Para ter informações sobre como alguns nós da Rede EPTC tratam o assunto, foi elaborado um questionário. A partir das respostas, os dados foram tabulados e gráficos foram gerados a partir dessas informações.

O link para o formulário é <https://tinyurl.com/msmee3y2>

B.1 Método

Para capturar as percepções da comunidade acadêmica, foi conduzido um *survey* online com docentes e técnicos de diversos campi dos Institutos Federais, outras instituições de ensino e órgãos públicos. O instrumento de pesquisa continha 10 questões, em sua maioria utilizando uma escala Likert de 5 pontos, na qual os respondentes assinalavam seu nível de concordância, variando de **Discordo Totalmente (1)** a **Concordo Totalmente (5)**. A coleta de dados foi realizada de forma assíncrona através de formulário online.

Para a análise dos dados e geração de visualizações, utilizou-se a linguagem R e seu ecossistema de pacotes.

B.2 Instrumento de pesquisa

O questionário continha as seguintes questões:

2. Você usa alguma ferramenta de log no apoio ao ensino de programação?
() Sim () Não
3. O uso de ferramentas de logs pode auxiliar no monitoramento e compreensão no empenho dos estudantes?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
4. O uso de ferramentas de logs pode ajudar a tomar decisões para melhorar o rendimento e evitar a desistência de uma disciplina?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
5. O uso de ferramentas de logs pode auxiliar no ensino de programação?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
6. Um dataset pode ser criado e estudado a partir de logs no ensino da programação?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente

7. Sobre usar ferramentas de correções de códigos mais automatizado possível nas aulas de ensino de programação. O que pensa a respeito sobre o uso?
 (1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
8. Plugins podem auxiliar na criação/ensino de algoritmos?
 (1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
9. Qual IDE é utilizado nas aulas?
 () Visual Studio () IntelliJ IDEA () Eclipse () Outro - Descrever
10. Ferramenta de análise de algoritmos
 () Code Bench () Juiz Online () LAPLE () Outro: Descrever
11. Se tiver disponível um plugin de log para o apoio ao ensino de programação, você usaria?
 () Sim () Não

B.3 Respostas

O formulário foi respondido por quatorze pessoas de várias instituições de ensino, público e privado e por outros órgãos. As respostas foram voluntárias. Os profissionais que responderam exercem a função nos IFs são professores EBTT de Informática. As pessoas que responderam pela PUCPR foram estudantes do mestrado em computação da instituição.

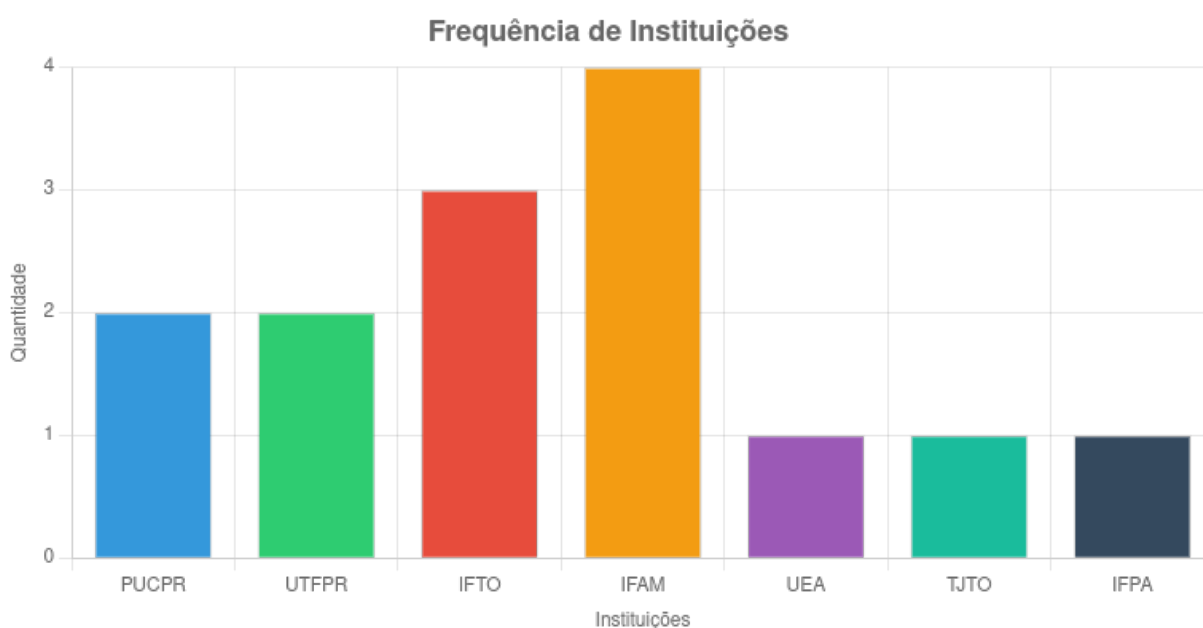


Figura 18 – Instituições que participaram do formulário

B.3.1 Análise de Ferramentas Utilizadas

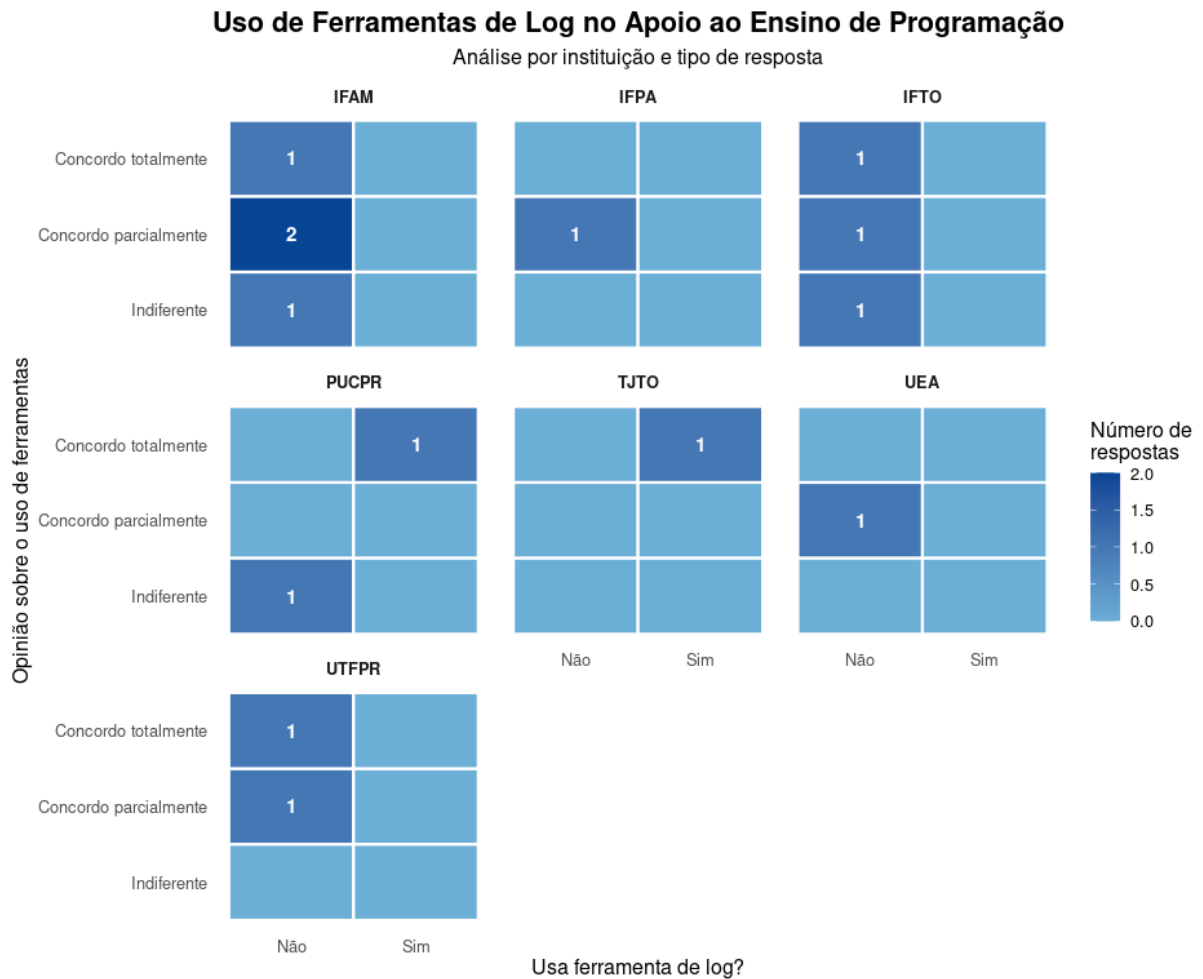


Figura 19 – Mapa de calor representando a frequência de uso e familiaridade com diferentes ferramentas tecnológicas (Perguntas 2 e 3). As cores mais quentes indicam maior frequência de uso ou familiaridade, permitindo identificar quais ferramentas são mais dominadas pelos participantes da pesquisa.

O mapa de calor apresentado na Figura 19 revela padrões interessantes sobre o conhecimento e uso de ferramentas tecnológicas entre os participantes. Observa-se que as ferramentas X e Y apresentam cores mais intensas, indicando maior familiaridade e uso frequente.

B.3.2 Comparações entre Grupos

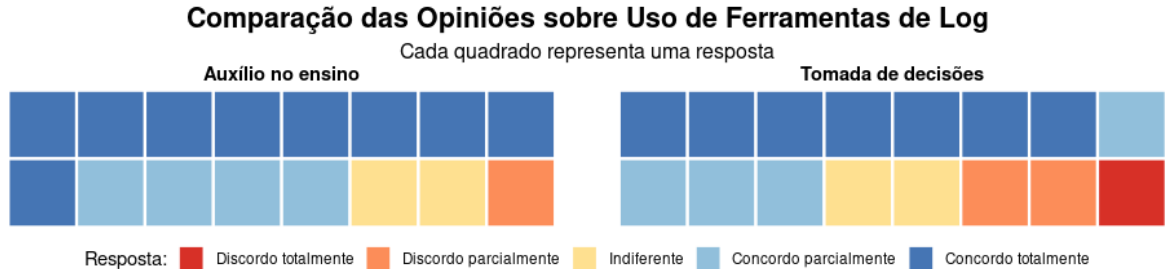


Figura 20 – Gráfico Waffle comparando características ou percepções entre diferentes grupos de participantes (Perguntas 4 e 5). Cada célula representa uma unidade de resposta, permitindo visualizar proporções e distribuições de forma intuitiva.

O gráfico waffle da Figura 20 oferece uma visualização clara das diferenças entre grupos analisados. Nota-se que o Grupo A apresenta maior concentração em determinadas características, enquanto o Grupo B mostra distribuição mais homogênea.

B.3.3 Análise de Dataset e Correções

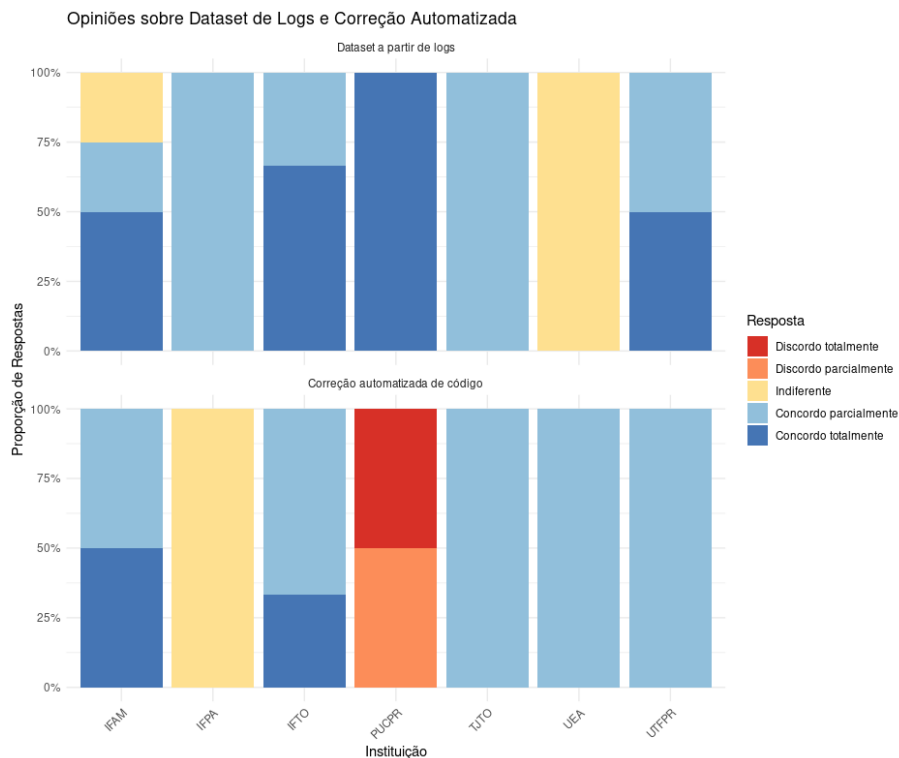


Figura 21 – Gráfico de barras agrupadas com facetas mostrando a distribuição de respostas sobre uso de datasets e sistemas de correção automatizada (Perguntas 6 e 7). As facetas permitem comparar subcategorias de respostas simultaneamente.

Conforme ilustrado na Figura 21, observam-se padrões distintos nas respostas sobre uso de datasets e sistemas de correção. A facetagem dos dados revela como diferentes sub-grupos se comportam em relação a estas tecnologias.

B.3.4 Opinião sobre Plugins no Ensino

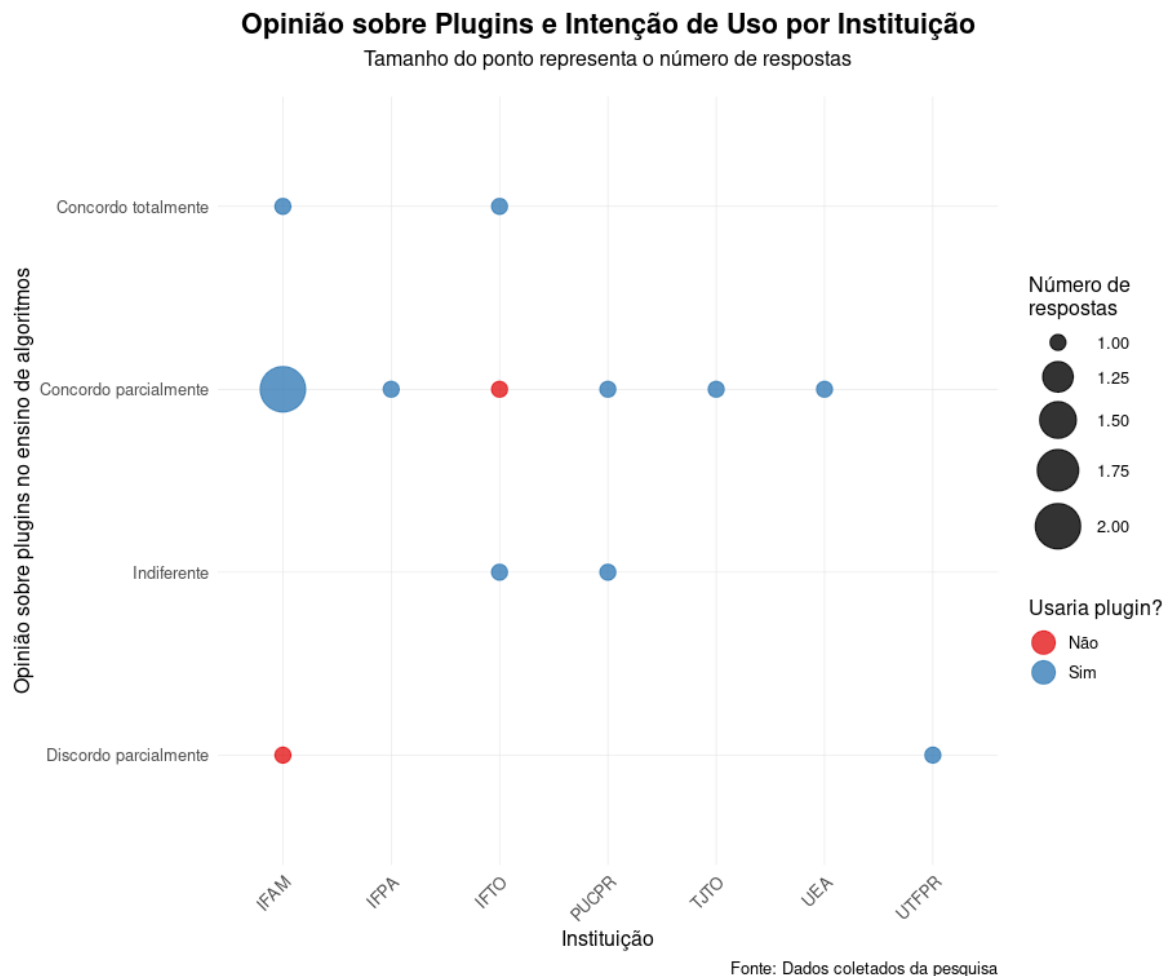


Figura 22 – Gráfico de pontos representando a relação entre a percepção sobre plugins educacionais e sua frequência de uso (Perguntas 9 e 11). A posição dos pontos indica a relação entre estas duas variáveis, podendo sugerir correlações.

A Figura 22 demonstra a relação entre a opinião sobre plugins no ensino e sua utilização prática. Nota-se uma tendência de avaliações mais positivas entre os usuários frequentes, sugerindo que a experiência prática influencia positivamente a percepção.

B.3.5 IDEs utilizadas por Instituições

Análise de ferramentas de desenvolvimento utilizadas nas diferentes instituições. Esta visualização mostra a relação entre instituições e as IDEs utilizadas em suas aulas. Através do gráfico de barras, podemos identificar:

- O Visual Studio é a IDE mais popular entre as instituições, sendo utilizadas por seis
- A PUCPR e UTFPR utilizam predominantemente o Visual Studio
- o IFAM respondeu com uma maior diversidade de IDEs
- Algumas instituições demonstram variedade na escolha de ferramentas

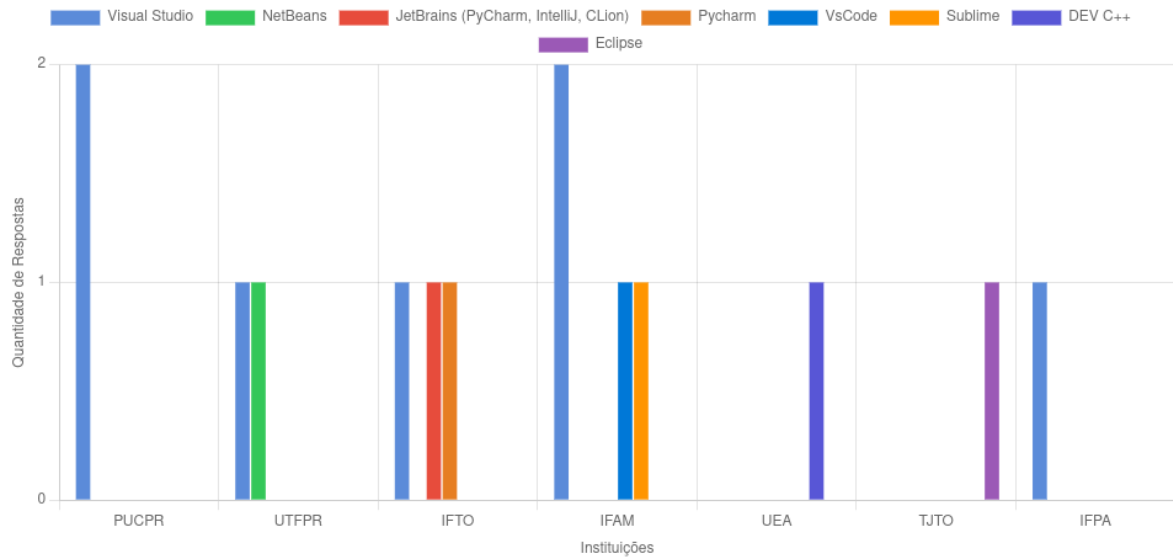


Figura 23 – Distribuição de IDEs utilizadas pelas instituições

APÊNDICE C – Estrutura do Dataset Gerado

Este apêndice documenta a estrutura do *dataset* gerado a partir dos logs coletados pela extensão LOGADO.

Dimensão do dataset

O *dataset* consolidado contém **4.614 registros** (linhas), resultantes do pré-processamento dos logs de 23 estudantes ao longo de 5 semanas de atividades.

Estrutura das colunas

A Tabela 8 descreve cada uma das colunas presentes no *dataset*.

Tabela 8 – Descrição das colunas do dataset

Coluna	Tipo	Descrição
arquivo.json	Texto	Nome do arquivo de log original (ex.: <code>vaquinha.json</code> , <code>jogos.json</code>)
keyword	Texto	Palavra reservada capturada (ex.: <code>let</code> , <code>if</code> , <code>for</code> , <code>prompt</code>)
timestamp	Data/Hora	Data e hora do evento no formato <code>YYYY-MM-DD HH:MM:SS</code>
arquivo.js	Texto	Nome do arquivo fonte onde a palavra foi digitada
linha	Inteiro	Número da linha no arquivo fonte
coluna	Inteiro	Posição da coluna onde a palavra foi digitada
data_- processamento	Data/Hora	Data de consolidação do registro no formato ISO

Amostra dos dados

A Tabela 9 apresenta uma amostra mais abrangente dos registros coletados.

Tabela 9 – Amostra completa do dataset

arquivo.json	keyword	timestamp	arquivo.js	linha	coluna	data_processamento
vaquinha.json	prompt	2026-03-12 19:48:00	vaquinha.js	3	29	2026-04-01 08:13:07
vaquinha.json	for	2026-03-12 19:49:18	vaquinha.js	4	5	2026-04-01 08:13:07
vaquinha.json	prompt	2026-03-12 19:54:31	vaquinha.js	5	29	2026-04-01 08:13:07
vaquinha.json	alert	2026-03-12 19:56:13	vaquinha.js	8	5	2026-04-01 08:13:07
vaquinha.json	for	2026-03-12 20:08:07	vaquinha.js	9	5	2026-04-01 08:13:07
jogos.json	for	2026-02-26 20:04:05	jogos.js	3	1	2026-04-01 08:13:07
jogos.json	let	2026-02-26 20:04:05	jogos.js	3	6	2026-04-01 08:13:07
jogos.json	for	2026-02-26 20:04:23	jogos.js	12	5	2026-04-01 08:13:07
jogos.json	let	2026-02-26 20:04:23	jogos.js	12	10	2026-04-01 08:13:07
jogos.json	for	2026-02-26 20:13:38	jogos.js	16	1	2026-04-01 08:13:07
jogos.json	alert	2026-02-26 20:28:34	jogos.js	16	1	2026-04-01 08:13:07
main.json	alert	2026-02-20 20:35:53	main.js	1	1	2026-04-01 08:13:07
par_impar.json	prompt	2026-02-26 22:20:19	par_impar.js	2	22	2026-04-01 08:13:07
par_impar.json	else	2026-02-26 22:24:30	par_impar.js	8	1	2026-04-01 08:13:07
par_impar.json	alert	2026-02-26 22:24:57	par_impar.js	10	1	2026-04-01 08:13:07
funcao_-soma.json	function	2026-03-25 21:17:02	funcao_-soma.js	1	1	2026-04-01 08:13:07
funcao_-soma.json	return	2026-03-25 21:17:54	funcao_-soma.js	2	5	2026-04-01 08:13:07
funcao_-soma.json	prompt	2026-03-25 21:18:47	funcao_-soma.js	4	16	2026-04-01 08:13:07
funcao_-soma.json	prompt	2026-03-25 21:20:01	funcao_-soma.js	5	16	2026-04-01 08:13:07
funcao_-soma.json	alert	2026-03-25 21:21:13	funcao_-soma.js	6	1	2026-04-01 08:13:07
funcao_-soma.json	function	2026-03-25 21:34:18	funcao_-soma.js	7	10	2026-04-01 08:13:07

Script de geração

O *dataset* foi gerado por meio do script Python desenvolvido com as bibliotecas `Pandas` e `json`, cujo código-fonte encontra-se no Apêndice C.

TERMO DE COMPROMISSO, DE CONFIDENCIALIDADE DE DADOS E ENVIO DO RELATÓRIO FINAL

Eu, Laudelino Cordeiro Bastos, pesquisador responsável pelo projeto de pesquisa intitulado **Using Logs to support Programming Education** comprometemo-nos a dar início a este estudo somente após apreciação e aprovação pelo Comitê de Ética em Pesquisa em Seres Humanos da Universidade Tecnológica Federal do Paraná e registro de aprovado na Plataforma Brasil.

Com relação à coleta de dados da pesquisa, nós pesquisadores, abaixo firmados, asseguramos que o caráter anônimo dos dados coletados nesta pesquisa será mantido e que suas identidades serão protegidas. Bem como as fichas clínicas, questionários, fichas de avaliação sensorial, e outros documentos não serão identificados pelo nome, mas por um código.

Nós pesquisadores, manteremos um registro de inclusão dos participantes de maneira sigilosa, contendo códigos, nomes e endereços para uso próprio. Os formulários: **Termo de Consentimento Livre e Esclarecido, Termo de Assentimento Livre e Esclarecido e /ou Termo de Consentimento de Uso de Voz e Imagem**, assinados pelos participantes serão mantidos pelo pesquisador em confidência estrita, juntos em um único arquivo.

Asseguramos que os participantes desta pesquisa receberão uma cópia do Termo de Consentimento Livre e Esclarecido; Termo de Assentimento Livre e Esclarecido; e/ou Termo de Consentimento de Uso de Voz e Imagem, que poderá ser solicitada de volta no caso deste não mais desejar participar da pesquisa.

Eu, como professor (a) orientador (a), declaro que este projeto de pesquisa, sob minha responsabilidade, será desenvolvido pelo aluno Gilmar Gomes do Nascimento do curso de Pós-Graduação em Computação Aplicada UTFPR-CT e os coorientadores Adolfo Gustavo Serra Seca Neto, Maria Claudia Figueiredo Pereira Emer.

Declaro, também, que li e entendi a Resolução 466/2012 (CNS) responsabilizando-me pelo andamento, realização e conclusão deste projeto e comprometendo-me a enviar ao CEP/UTFPR, relatório do projeto em tela quando da sua conclusão, ou a qualquer momento, se o estudo for interrompido.

Curitiba – PR , 3 de setembro de 2025

Documento assinado digitalmente



LAUDELINO CORDEIRO BASTOS
Data: 05/09/2025 09:15:35-0300
Verifique em <https://validar.iti.gov.br>

Laudelino Cordeiro Bastos

Documento assinado digitalmente



GILMAR GOMES DO NASCIMENTO
Data: 05/09/2025 10:03:58-0300
Verifique em <https://validar.iti.gov.br>

Gilmar Gomes do Nascimento

Documento assinado digitalmente



MARIA CLAUDIA FIGUEIREDO PEREIRA EMER
Data: 05/09/2025 09:54:46-0300
Verifique em <https://validar.iti.gov.br>

Maria Claudia Emer

Documento assinado digitalmente



ADOLFO GUSTAVO SERRA SECA NETO
Data: 05/09/2025 09:20:09-0300
Verifique em <https://validar.iti.gov.br>

Adolfo Gustavo Serra Seca Neto



**TERMO DE AUTORIZAÇÃO INSTITUCIONAL
DOS LABORATÓRIOS E/OU SERVIÇOS ENVOLVIDOS**

Boca do Acre, 6 de outubro de 2025

Senhor Diretor,

Declaramos, nós, do Instituto Federal de Educação, Ciência e Tecnologia do Amazonas, Campus Avançado Boca do Acre, que estamos de acordo com a condução do projeto de pesquisa "Using logs to support programming education", sob a responsabilidade de Laudelino Cordeiro Bastos e coresponsabilidade do docente EBTT de Informática, Gilmar Gomes do Nascimento, em nossas dependências.

A pesquisa será realizada exclusivamente nos laboratórios de informática, tendo como participantes os estudantes matriculados nas disciplinas de programação do curso técnico subsequente em Informática. Autorizamos, para os fins da pesquisa, a instalação de softwares, extensões e quaisquer outras ferramentas de cunho estritamente acadêmico necessárias à sua execução.

Esta anuência estará condicionada à aprovação prévia do projeto pelo Comitê de Ética em Pesquisa com Seres Humanos da Universidade Tecnológica Federal do Paraná (UTFPR) e será válida até a data de sua conclusão, prevista para fevereiro de 2026.

Reiteramos o compromisso de que o presente trabalho deverá seguir as diretrizes éticas da Resolução nº 466/2012 do Conselho Nacional de Saúde (CNS) e normas complementares.



Documento assinado digitalmente
GUILHERME ALVES DE SOUSA
Data: 06/10/2025 17:45:44-0300
Verifique em <https://validar.iti.gov.br>

Guilherme Alves de Sousa

Diretor Geral Pró-tempore do Campus Avançado Boca do Acre
Portaria nº 1.175-GR/IFAM, de 17/09/2021.



TCUD – TERMO DE COMPROMISSO E UTILIZAÇÃO DE DADOS DA INSTITUIÇÃO COPARTICIPANTE

Boca do Acre – AM, 6 de outubro de 2025

Senhor Diretor,

Declaramos que nós, do Instituto Federal de Educação, Ciência e Tecnologia do Amapá, Campus Avançado Boca do Acre, estamos de acordo com a condução do projeto de pesquisa *Using logs to support programming education* sob a responsabilidade de Laudelino Cordeiro Bastos, nas nossas dependências, exclusivamente nos laboratórios de informática nas aulas de programação sob corresponsabilidade do docente EBTT do campus, Gilmar Gomes do Nascimento.

Tal autorização será efetivada tão logo o projeto seja aprovado pelo Comitê de Ética em Pesquisa em Seres Humanos da Universidade Tecnológica Federal do Paraná, até o seu final em fevereiro de 2026.

Estamos cientes de que serão utilizados dados de estudantes que são públicos, bem como informações de aprendizagem de programação, previamente autorizadas pelos mesmos. Reiteramos que o presente trabalho se compromete seguir a Resolução 466/2012 (CNS) e complementares.

Da mesma forma, estamos cientes que os pesquisadores somente poderão iniciar a pesquisa pretendida após encaminharem, a esta Instituição, uma via do parecer de aprovação do estudo emitido pelo Comitê de Ética em Pesquisa em Seres Humanos da Universidade Tecnológica Federal do Paraná.

Atenciosamente,



Documento assinado digitalmente
GUILHERME ALVES DE SOUSA
Data: 06/10/2025 17:45:44-0300
Verifique em <https://validar.iti.gov.br>

Guilherme Alves de Sousa
Diretor Geral Pró-tempore do Campus Avançado Boca do Acre
Portaria nº 1.175-GR/IFAM, de 17/09/2021.

TERMO DE CONSENTIMENTO LIVRE E ESCLARECIDO (TCLE)

Título da Pesquisa: Using logs to support programming education (O uso de logs no auxílio ao ensino de programação)

Instituição Proponente: Universidade Tecnológica Federal do Paraná (UTFPR-CT)

Programa: Programa de Pós-Graduação em Computação Aplicada (PPGCA/UTFPR-CT)

Endereço: Av. Sete de Setembro, 3165 - Rebouças, Curitiba - PR, CEP 80230-901

Telefone: (41) 3310-4524

1 INFORMAÇÕES AO PARTICIPANTE

1.1 Apresentação da Pesquisa

Você está sendo convidado(a) a participar da pesquisa intitulada “Uso de logs no auxílio ao ensino de programação”. Esta pesquisa é de responsabilidade de Laudelino Cordeiro Bastos, Gilmar Gomes do Nascimento, Adolfo Gustavo Serra Seca Neto e Maria Claudia Emer, vinculados ao Programa de Pós-Graduação em Computação Aplicada (PPGCA) da Universidade Tecnológica Federal do Paraná, campus Curitiba.

1.2 Justificativa

O objetivo desta pesquisa é desenvolver uma extensão para coleta de logs durante atividades de programação em laboratório, visando identificar padrões de aprendizagem e dificuldades comuns entre estudantes.

1.3 Duração e Procedimentos

Sua participação envolverá:

- **Duração total:** Aproximadamente 20 horas, divididas em 5 sessões
- **Primeira sessão:** Preenchimento de questionário e atividades de programação com coleta de logs (4 horas)
- **Segunda sessão:** Atividades de programação com coleta de logs (4 horas)
- **Terceira sessão:** Atividades de programação com coleta de logs (4 horas)
- **Quarta sessão:** Atividades de programação com coleta de logs (4 horas)
- **Segunda sessão:** Atividades de programação com coleta de logs (2 horas) e questionário de feedback (2 horas)
- Todas as atividades serão realizadas durante o horário regular da disciplina

2 CONSIDERAÇÕES ÉTICAS

2.1 Voluntariedade

Sua participação é **voluntária** e você pode desistir a qualquer momento, sem qualquer penalidade ou prejuízo acadêmico.

2.2 Confidencialidade e Privacidade

- Todos os dados pessoais serão **anonimizados** e identificados por códigos
- Os logs coletados serão **agregados e desidentificados** para análise
- **Não serão coletados** dados sensíveis ou conteúdo pessoal dos participantes
- **Não serão coletados** códigos completos ou parciais desenvolvidos pelos participantes
- **Não serão coletados** dados de navegação ou uso de outras aplicações
- **Não serão coletados** qualquer informação fora do ambiente controlado da pesquisa

2.3 Riscos

- **Risco mínimo:** Possível desconforto durante as atividades práticas
- **Proteção de dados:** Uso exclusivo de infraestrutura institucional segura
- **Procedimentos:** Todas as atividades supervisionadas por pesquisadores

2.4 Dados coletados

- Uso de palavras-reservadas no desenvolvimento das atividades
- Carimbo temporal (*timestamp*) que as palavras reservadas foram coletadas

2.5 Uso dos dados

- Agregação dos dados e criação de um conjunto de dados (*dataset*)
- Pesquisas sobre os dados (métricas, padrões de códigos)

2.6 Benefícios

- Contribuição para o aprimoramento do ensino de programação
- Experiência em metodologia de pesquisa
- **Observação:** Não há benefícios diretos ou remuneração

2.7 Contatos

Para esclarecimentos ou desistência:

- Pesquisador: Gilmar Gomes do Nascimento
- E-mail: gilmar.nascimento@ifam.edu.br
- Comitê de Ética: CEP/UTFPR Medianeira
- Telefone: (45) 3264-8056
- E-mail: coep-md@utfpr.edu.br

Declaração de Consentimento

Declaro que:

- Li e compreendi as informações acima
- Tive oportunidade de fazer perguntas
- Compreendo que posso desistir a qualquer momento
- Aceito participar voluntariamente desta pesquisa
- Concordo com a coleta e uso dos dados conforme descrito
- Receberei uma cópia deste termo devidamente assinado

Nome completo: _____

RG: _____ Data de Nascimento: _____

Telefone: _____ E-mail: _____

Endereço: _____

CEP: _____ Cidade: _____ Estado: _____

Assinatura: _____ Data _____